

Prompt: I am cleaning up the questions I used while working through the build, but I want them to stay close to how I actually think about the project. I want to start from the idea rather than jump straight into the verifier. I was thinking about a campus delivery system instead of another warehouse example - can you help me turn that into one coherent autonomous system?

Response: That framing works. I would treat the conversation as a development aid and keep the technical record tied to the project files and observed runs. The campus story can be the starting point, then the build can grow around the required pieces rather than becoming six disconnected demos. I would also keep the coursework boundary visible throughout: the older lab algorithms stay identified as reused/adapted code, while the new CampusCourier work is the application and integration layer. I would not present the cleaned-up wording as a verbatim transcript.

Prompt: I have the campus idea, but I do not want to bolt on random features just because they sound good. Which parts of the story should actually affect a decision in the code, and which parts are basically just there for the scenario?

Response: A useful way to separate the features is to ask whether removing one changes a program decision. Battery, capacity, weights, stock, and the quiet-zone masking can change what the system does. The wall-count sensor is used by the HMM heatmap, but the belief does not currently drive another decision. The mopped floor, Corridor B conflict story, and the phrase “before the 2pm class” do not currently change the dispatch decision because the adaptive, conflict, and time/deadline mechanics are not integrated.

That is important for how I describe the project: the dashboard marks the adaptive pilot and Corridor B conflict as planned, and the deadline is not modelled in code. I should not describe those pieces as if they are active mechanics.

Prompt: I already have working coursework code, and the project rules say I can reuse it but cannot have an LLM rewrite the essential algorithms. How can I use AI as a helper while I build around that code without crossing that line?

Response: The clean boundary is to keep the coursework algorithms as the protected core and use LLM help around them for the new application layer. In the current project, the Lab 1 A* files and the submitted Lab 2 filter can be checked against their originals, while the CampusCourier-specific integration lives mainly in `world.py`, `perception.py`, `memory.py`, `agent.py`, `app.py`, and the project tests. The LLM can help explain those old algorithms, reason about interfaces, and help with integration, UI, debugging, and testing without turning the old lab implementations into regenerated code. The important thing is that I can show the diff and explain the boundary rather than just saying I reused my labs.

Prompt: I keep saying I built CampusCourier from scratch, but I know some of the algorithms came from my coursework. What is the most honest way to explain what I built myself versus what I reused?

Response: I would say: **“I built the CampusCourier application and integration layer from scratch around reused coursework algorithms.”** The new system-level work includes the campus scenario, the SQLite schema and state model, the robot/supply data, the project-specific tools, the grounding and commit gates, the route-policy adapter, the HMM-to-campus adapter, the scripted end-to-end stories, the stress tests, the dashboard, the logging, and the integration between all of those pieces. The algorithms are not the part I need to pretend I recreated. Reusing a correct A* implementation is exactly what the project rule allows. The substantial work is making that A* implementation operate on a different map and then making its result participate in a much larger agent pipeline without letting the LLM bypass the deterministic parts.

Prompt: Once I decided not to rewrite the old algorithms, what new pieces did I actually have to build around them to turn them into CampusCourier rather than just copying the lab files over?

Response: The hardest part was getting interfaces with different assumptions to agree without rewriting the underlying lab algorithms. The clearest examples are coordinate conventions between `world.py` and Lab 2, converting the campus ASCII map into the Lab 1 environment format, converting LLM JSON into typed Pydantic models, translating solver assignments back from integer AGV indices to robot IDs, and keeping database state synchronized with the result of a successful run. The solution was to keep the existing algorithm modules stable and put the translation logic in thin adapters. `world.py` is a route adapter, `perception.py` is an HMM adapter, and `agent.py` is the orchestration boundary. That separation also makes the system easier to explain because each file has a relatively narrow reason to exist.

Prompt: I was trying to get the important parts working first and leave the optional ideas for later. Can you help

me look at what I actually cut and how that affected the final system?

Response: You cut the adaptive pilot (MDP and Q-learning), Corridor B self-play, the return-trip reserve, time and deadlines, and the quiet zone as a Z3 rule. The important part is that the current code makes the status visible instead of quietly implying that every layer is finished.

app.py, lines 36-44

```
36 LAYERS = [  
37     ("LLM coordinator + Z3 safety filter", "live"),  
38     ("A* routing", "live"),  
39     ("SQLite memory", "live"),  
40     ("Live dashboard", "live"),  
41     ("HMM localisation (Lab 2 filter)", "live"),  
42     ("Adaptive pilot (MDP + RL)", "planned"),  
43     ("Corridor B conflict (self-play)", "planned"),  
44 ]
```

So the working system was prioritised around the coordinator, deterministic verification, routing, persistence, the perception demonstration and the dashboard. The adaptive and conflict pieces remain explicitly planned instead of being represented as complete. The proposal also records the intention to prioritise the mandatory verifier first. The existing value-iteration lab was already available for later adaptation, while extra instrumentation such as W&B logging and the stress suite also took development time.

Prompt: I wanted the three robots to behave differently so the examples would expose different constraints. Can you help me understand what I was trying to demonstrate with Atlas, Comet and Pixel?

Response: Atlas is the workhorse that makes plans possible. Comet is strong enough to carry useful loads but is battery-tight and does not have quiet wheels. Pixel is deliberately below its minimum battery, so it gives the system a robot that must be refused.

memory.py, lines 44-49

```
bx, by = LOCATIONS["Supply Room"]  
ROBOTS = [  
    ("CB-1", "Atlas", 30, 85, 20, 0.5, 1, bx, by),  
    ("CB-2", "Comet", 20, 40, 20, 1.0, 0, bx + 1, by),  
    ("CB-3", "Pixel", 10, 15, 20, 0.3, 1, bx, by + 1),  
]
```

Atlas has 30 kg capacity and 85% battery, so capacity is the main limitation in the routine examples. Comet has 20 kg capacity and starts at 40%, exactly enough that one 12 kg projector leaves 28%, but another would leave 16%, below the 20% minimum. Pixel starts at 15%, below its 20% minimum, so the verifier cannot assign it any package.

The important idea is that the three robots are not cosmetic variations: their values create different test cases. Comet demonstrates that a load can fit by capacity but still fail because of battery drain. Pixel demonstrates a robot that is available in the fleet data but ineligible for dispatch.

One limitation is worth keeping visible: Pixel's quiet wheels never matter because Pixel cannot move at all, and Comet's lack of quiet wheels does not change its route to Studio 3 in the main demo.

Prompt: I also chose some very specific item weights. Why are the 12 kg projectors and 18 kg whiteboard useful for the examples instead of just being arbitrary numbers?

Response: 12 kg is just over Pixel's limit and fits twice into Atlas. 18 kg fits Comet but pushes the overload order to 54 kg, against 50 kg of usable capacity. Together they make a refusal that a human can check on the back of an envelope.

memory.py, lines 50-56 50 SUPPLIES = [51 ("projector", 12, 6), 52 ("lab_kit", 5, 4), 53 ("water_case", 8, 5), 54 ("whiteboard", 18, 1), 55 ("marker_box", 2, 10), 56]

The arithmetic each number enables: • Routine (2 projectors + lab kit = 29 kg). It fits on Atlas alone ($29 \leq 30$), which is exactly what Z3 picked in the demo run. Battery: $85 - 0.5 \times 29 = 70.5$, stored as 70. Z3 just returns the first model it finds, so it does not prefer one robot or spread the load; with Pixel in the payload it happened to split

the same order Atlas 12 / Comet 17 instead. • Overload (3 projectors + whiteboard = 54 kg). The usable fleet is Atlas + Comet = 50 kg, so the order is UNSAT on total capacity, which is the easiest UNSAT to explain to a room. • Whiteboard stock = 1. Only one can ever be requested, so the grounding gate refuses two (agent.py:88-90). • Lab kit 5, water case 8, marker box 2. Small fillers, so that partial plans exist and the model has something to trade. One thing I need to flag. “Drop the whiteboard” is a policy choice, not a mathematical necessity. 2 projectors + the whiteboard (42 kg) is also feasible: Atlas takes both projectors (24 kg), Comet takes

the whiteboard ($40 - 18 = 22 \geq 20$). The script drops the whiteboard because the prompt’s priority list puts projectors first (agent.py:134-135). Say “the maths says something must go; the priority rule decides what”. Don’t say “Z3 told it to drop the whiteboard”.

Prompt: I have the idea and the coursework pieces now. What would be a sensible way to turn that into actual buildable layers, and which files should I expect to create around the old algorithms?

Response: I would start with `agent.py` and `run_agent()` because that is the spine of the application. Then I would trace into `memory.py` for the persistent state, `world.py` for the campus-to-A* adapter, `core/verifier.py` for the Z3 model, and `app.py` for the dashboard. After that I would run a normal dispatch and the overload case so I can see the same pieces interact: the LLM proposes, the tools return facts, A* plans the route, Z3 checks the allocation, the finish gate checks the final transition, and SQLite records the committed state. Finally I would look at the tests. That gives me a build-oriented view of the system before getting into the detailed verification audit.

Prompt: I want to use AI as a development helper, not as a replacement for the work I am supposed to do myself. Looking at this project, where is LLM help actually useful - framing the architecture, integration, debugging, UI, ReAct, testing - while the old algorithms stay mine?

Response: I would answer by separating **algorithm source**, **integration source**, and **LLM-facing source**. For algorithm source, the repository comparison shows the Lab 1 planner, environment, and models are unchanged, the HMM filter contains the specific student TODO completions, and the SMT verifier contains the two required corrections. Those are the pieces I treated as coursework-derived working code. For integration source, I used LLM assistance extensively to reason through how to structure `agent.py`, `world.py`, `perception.py`, `memory.py`, `app.py`, testing scenarios, and the dashboard. I also used small snippets and debugging suggestions where they helped me move between interfaces. For LLM-facing source, the project intentionally calls an external LLM for the natural-language command and ReAct decision loop. That use is not an accidental by-product; it is one of the project’s required concepts. So I would not say “the LLM wrote nothing.” I would say **“the LLM was an active development aid and is part of the runtime coordinator, but the essential coursework algorithms were carried forward as my existing lab implementations and corrected/adapted rather than replaced wholesale.”**

Prompt: I want an offline mode because I do not want the project to stop working every time the API or Wi-Fi is unavailable. What should the scripted engine actually do, and which parts of my real pipeline should still run?

Response: The offline engine is not pretending to be an LLM. It is a deterministic replay mechanism for testing and demos. The important point is that it still goes through the real `run_agent()` implementation. The scripted object returns the same kind of JSON that the live model would return. After that, the real project code performs Pydantic validation, grounding checks, Z3 verification, A* route planning, the finish/commit gate, and SQLite persistence. In the overload story the scripted step even waits for the real Z3 observation before constructing its final `finish` call. So offline mode removes only the uncertain external model call. It does not replace the deterministic safety machinery. That makes it useful for demonstrating the verifier behaviour without requiring wifi or an API key.

```
class ScriptedLLM:
    """Offline replay (same idea as the ReAct lab's scripted_llm.py). A step may be a
    callable taking the last observation, so it can copy Z3's assignment."""
    label = "Offline scripted replay (no LLM)"

    def __init__(self, steps):
        self.steps, self.i = steps, 0

    def step(self, messages):
        step = self.steps[self.i]
        self.i += 1
```

```

        if callable(step):
            step = step(json.loads(messages[-1]["content"].removeprefix("Observation: ")))
        return json.dumps(step)

def scripted_delivery(overloaded=False):
    fleet = [r for r in memory.robots() if r["battery_pct"] >= r["min_battery"]]
    specs = [{k: r[k] for k in ("capacity", "battery_pct", "min_battery", "drain_rate")} | {"id": r["robot_id"]}
              for r in fleet]
    stock = {s["item"]: s["weight"] for s in memory.supplies()}

    def pkgs(**counts):
        return [{"id": f"{item}#{n}", "weight": stock[item]} for item, c in counts.items() for n in range(1, c+1)]

    steps = [
        {"thought": "Check which robots are usable.", "action": "get_fleet_status", "action_input": {}},
        {"thought": "Check supply weights and stock.", "action": "get_inventory", "action_input": {}},
    ]
    steps += [{"thought": f"Route {s['id']} to Studio 3.", "action": "plan_route",
               "action_input": {"robot_id": s["id"], "destination": "Studio 3"}} for s in specs]
    if overloaded:
        steps.append({"thought": "Try the full request: 3 projectors and the whiteboard.",
                      "action": "verify_dispatch_smt",
                      "action_input": {"packages": pkgs(projector=3, whiteboard=1), "agvs": specs}})
        request, note = pkgs(projector=3), "The whiteboard was dropped: total load exceeded usable capacity."
        thought = "UNSAT on capacity. The class needs the projectors, so drop the whiteboard and retry."
    else:
        request, note, thought = pkgs(projector=2, lab_kit=1), "", "Verify 2 projectors + 1 lab kit."
    steps.append({"thought": thought, "action": "verify_dispatch_smt",
                  "action_input": {"packages": request, "agvs": specs}})
    steps.append(lambda obs: {
        "thought": "Z3 verified the plan; commit it." if obs.get("status") == "SATISFIABLE" else "Still inferring.",
        "action": "finish",
        "action_input": {"final_answer": f"Dispatched {len(request)} items to Studio 3.{note}"
                        if obs.get("status") == "SATISFIABLE" else "Order cannot be fulfilled safely.",
                        "destination": "Studio 3", "assignments": obs.get("assignments", {})},
    })
    return ScriptedLLM(steps)

```

Prompt: Before I get too deep into the agent, I want to get the persistent state right. What should the database own in this project, and why is that better than keeping the fleet state inside the LLM conversation?

Response: The `robots` table is the current fleet state: identity, capacity, battery, minimum battery, drain rate, quiet-wheel capability, and position. `supplies` is the current inventory: item name, unit weight, and stock. `tasks` is the dispatch history: when the task happened, the original order text, the robot, items, destination, route length, and status. The rest of the application reads those tables through `robots()` and `supplies()`, while `record_dispatch()` writes the task row and updates current state.

```

SCHEMA = """
CREATE TABLE robots (
    robot_id    TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    capacity    INTEGER NOT NULL,    -- kg
    battery_pct INTEGER NOT NULL,
    min_battery INTEGER NOT NULL,
    drain_rate  REAL NOT NULL,      -- battery % used per kg carried
    quiet_wheels INTEGER NOT NULL,  -- 1 = allowed through the quiet zone
    x           INTEGER NOT NULL, y INTEGER NOT NULL
)

```

```

);
CREATE TABLE supplies (
    item    TEXT PRIMARY KEY,
    weight  INTEGER NOT NULL,      -- kg per unit
    stock   INTEGER NOT NULL
);
CREATE TABLE tasks (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT NOT NULL DEFAULT (datetime('now', 'localtime')),
    order_text TEXT NOT NULL,
    robot_id  TEXT NOT NULL,
    items     TEXT NOT NULL,
    destination TEXT NOT NULL,
    route_steps INTEGER NOT NULL,
    status    TEXT NOT NULL
);
"""

```

Prompt: When I started joining the coursework pieces together, I was worried that tiny interface differences would break everything. Can you point out the important mismatches between the old code and CampusCourier and how I can isolate them in adapters?

Response: The hardest part was getting interfaces with different assumptions to agree without rewriting the underlying lab algorithms. The clearest examples are coordinate conventions between `world.py` and Lab 2, converting the campus ASCII map into the Lab 1 environment format, converting LLM JSON into typed Pydantic models, translating solver assignments back from integer AGV indices to robot IDs, and keeping database state synchronized with the result of a successful run. The solution was to keep the existing algorithm modules stable and put the translation logic in thin adapters. `world.py` is a route adapter, `perception.py` is an HMM adapter, and `agent.py` is the orchestration boundary. That separation also makes the system easier to explain because each file has a relatively narrow reason to exist.

Prompt: I keep seeing `memory.reset()` in the tests. Does resetting the database defeat the whole point of persistence, or is it just there to give each test a clean starting state?

Response: Reset is a test and demo boundary, not a normal dispatch operation. It creates a known starting state so an overload test always begins with the same three robots and inventory. During a normal run, the state is not rebuilt; `record_dispatch()` modifies the existing database. This distinction lets me test deterministic scenarios while still having persistent state.

]

```

def _connect():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def reset():
    DB_PATH.unlink(missing_ok=True)
    with closing(_connect()) as conn, conn: # closes the file handle and commits
        conn.executescript(SCHEMA)
        conn.executemany("INSERT INTO robots VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", ROBOTS)
        conn.executemany("INSERT INTO supplies VALUES (?, ?, ?)", SUPPLIES)

def _rows(sql):
    if not DB_PATH.exists():
        reset()

```

```

with closing(_connect()) as conn, conn: # closes the file handle and commits
    return [dict(r) for r in conn.execute(sql)]

```

```

def robots():
    return _rows("SELECT * FROM robots ORDER BY robot_id")

```

```

def supplies():
    return _rows("SELECT * FROM supplies ORDER BY item")

```

```

def tasks():

```

Prompt: Can you walk through a successful record_dispatch() for me? I want to understand exactly how the task log, battery, robot position and inventory all change and why they are committed together.

Response: The function opens one SQLite connection, processes the assigned robots, inserts the corresponding task records, updates their battery and position, then decrements inventory. Those operations are committed together. The point is consistency: I do not want the database to show a robot as having completed a task while its inventory update failed, or vice versa.

```

    return _rows("SELECT * FROM tasks ORDER BY id DESC")

```

```

def record_dispatch(order_text, destination, packages, assignments, routes):
    """Commit a verified plan: log tasks, draw down stock, drain battery, move robots."""
    weights = {p["id"]: p["weight"] for p in packages}
    fleet = {r["robot_id"]: r for r in robots()}
    goal = LOCATIONS[destination]
    with closing(_connect()) as conn, conn: # closes the file handle and commits
        for robot_id in sorted(set(assignments.values())):
            items = [pid for pid, rid in assignments.items() if rid == robot_id]
            load = sum(weights[i] for i in items)
            new_battery = int(fleet[robot_id]["battery_pct"] - fleet[robot_id]["drain_rate"] * load)
            conn.execute(
                "INSERT INTO tasks (order_text, robot_id, items, destination, route_steps, status) "
                "VALUES (?, ?, ?, ?, ?, 'dispatched')",
                (order_text, robot_id, json.dumps(items), destination, routes[robot_id]["steps"]),
            )
            conn.execute("UPDATE robots SET battery_pct = ?, x = ?, y = ? WHERE robot_id = ?",
                          (new_battery, goal[0], goal[1], robot_id))
        for pid in assignments:
            conn.execute("UPDATE supplies SET stock = stock - 1 WHERE item = ?", (pid.split("#")[0],))

```

Prompt: The dashboard needs the full A* path, but the database only keeps route_steps. Why did I keep those two representations separate?

Response: The full path remains in the in-memory run["routes"] result so the UI can draw it during that run. The persistent task table records the compact operational metric route_steps. That keeps the historical schema simple while still letting the current dashboard show the detailed path.

```

    return _rows("SELECT * FROM tasks ORDER BY id DESC")

```

```

def record_dispatch(order_text, destination, packages, assignments, routes):
    """Commit a verified plan: log tasks, draw down stock, drain battery, move robots."""
    weights = {p["id"]: p["weight"] for p in packages}
    fleet = {r["robot_id"]: r for r in robots()}
    goal = LOCATIONS[destination]

```

```

with closing(_connect()) as conn, conn: # closes the file handle and commits
    for robot_id in sorted(set(assignments.values())):
        items = [pid for pid, rid in assignments.items() if rid == robot_id]
        load = sum(weights[i] for i in items)
        new_battery = int(fleet[robot_id]["battery_pct"] - fleet[robot_id]["drain_rate"] * load)
        conn.execute(
            "INSERT INTO tasks (order_text, robot_id, items, destination, route_steps, status) "
            "VALUES (?, ?, ?, ?, ?, 'dispatched')",
            (order_text, robot_id, json.dumps(items), destination, routes[robot_id]["steps"]),
        )
        conn.execute("UPDATE robots SET battery_pct = ?, x = ?, y = ? WHERE robot_id = ?",
                     (new_battery, goal[0], goal[1], robot_id))
    for pid in assignments:
        conn.execute("UPDATE supplies SET stock = stock - 1 WHERE item = ?", (pid.split("#")[0],))

```

Prompt: I understand why I need a database technically, but what failure would I actually get if the LLM were allowed to remember battery and stock values itself?

Response: Without external state, the model would become the source of truth for facts that are supposed to be authoritative. That creates a very obvious failure mode: after a rejection, the model could “repair” the proposal by saying that Pixel is actually at 95% battery. With SQLite, the agent’s tool observations come from the database, and `grounding_problems()` can compare any numbers that come back in a proposed verifier payload against those values. The database also survives the end of a single LLM context, so the next order sees the batteries, inventory, and task history left by the previous committed dispatch. In other words, SQLite is not there because a three-robot toy needs a database. It is there because the architecture needs a state boundary that the model cannot rewrite through prose.

```

with closing(_connect()) as conn, conn: # closes the file handle and commits
    conn.executescript(SCHEMA)
    conn.executemany("INSERT INTO robots VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)", ROBOTS)
    conn.executemany("INSERT INTO supplies VALUES (?, ?, ?)", SUPPLIES)

def _rows(sql):
    if not DB_PATH.exists():
        reset()
    with closing(_connect()) as conn, conn: # closes the file handle and commits
        return [dict(r) for r in conn.execute(sql)]

def robots():
    return _rows("SELECT * FROM robots ORDER BY robot_id")

def supplies():
    return _rows("SELECT * FROM supplies ORDER BY item")

def tasks():
    return _rows("SELECT * FROM tasks ORDER BY id DESC")

def record_dispatch(order_text, destination, packages, assignments, routes):
    """Commit a verified plan: log tasks, draw down stock, drain battery, move robots."""
    weights = {p["id"]: p["weight"] for p in packages}
    fleet = {r["robot_id"]: r for r in robots()}
    goal = LOCATIONS[destination]
    with closing(_connect()) as conn, conn: # closes the file handle and commits

```

```

for robot_id in sorted(set(assignments.values())):
    items = [pid for pid, rid in assignments.items() if rid == robot_id]
    load = sum(weights[i] for i in items)
    new_battery = int(fleet[robot_id]["battery_pct"] - fleet[robot_id]["drain_rate"] * load)
    conn.execute(
        "INSERT INTO tasks (order_text, robot_id, items, destination, route_steps, status) "
        "VALUES (?, ?, ?, ?, ?, 'dispatched')",
        (order_text, robot_id, json.dumps(items), destination, routes[robot_id]["steps"]),
    )
    conn.execute("UPDATE robots SET battery_pct = ?, x = ?, y = ? WHERE robot_id = ?",
                 (new_battery, goal[0], goal[1], robot_id))
for pid in assignments:
    conn.execute("UPDATE supplies SET stock = stock - 1 WHERE item = ?", (pid.split("#")[0],))

```

Prompt: I want to be able to explain how much of this work was AI-assisted without pretending the LLM wrote my coursework algorithms. Can you separate the reused algorithm work from the new integration, UI, debugging and ReAct work?

Response: I would answer by separating **algorithm source**, **integration source**, and **LLM-facing source**. For algorithm source, the repository comparison shows the Lab 1 planner, environment, and models are unchanged, the HMM filter contains the specific student TODO completions, and the SMT verifier contains the two required corrections. Those are the pieces I treated as coursework-derived working code. For integration source, I used LLM assistance extensively to reason through how to structure `agent.py`, `world.py`, `perception.py`, `memory.py`, `app.py`, testing scenarios, and the dashboard. I also used small snippets and debugging suggestions where they helped me move between interfaces. For LLM-facing source, the project intentionally calls an external LLM for the natural-language command and ReAct decision loop. That use is not an accidental by-product; it is one of the project's required concepts. So I would not say "the LLM wrote nothing." I would say **"the LLM was an active development aid and is part of the runtime coordinator, but the essential coursework algorithms were carried forward as my existing lab implementations and corrected/adapted rather than replaced wholesale."**

Prompt: I am trying to understand the live LLM call rather than treating it like a black box. What exactly does `OpenAILLM.step()` send on each turn, and why does it keep the previous messages?

Response: It sends the fixed `SYSTEM_PROMPT` first and then the accumulated `messages` list. The system prompt defines the output protocol, available actions, and safety instructions. The message history contains the original order, previous model JSON, and observations produced by tools. Keeping the history is what lets the model make a new decision based on what happened earlier in the same run.

```

class OpenAILLM:
    label = "OpenAI (live)"

    def __init__(self, model=None):
        from openai import OpenAI
        self.client = OpenAI(timeout=60, max_retries=1) # fail fast on bad wifi instead of hanging the den
        self.model = model or os.getenv("OPENAI_MODEL", "gpt-5-mini")

    def step(self, messages):
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "system", "content": SYSTEM_PROMPT}] + messages,
            response_format={"type": "json_object"},
        )
        return response.choices[0].message.content

```

Prompt: I used the ReAct lab as a starting point, but CampusCourier is a different problem. Can you walk me through what I actually kept from the old ReAct loop and what I changed for this project?

Response: The sequence is preserved almost exactly: raw model string -> JSON parse -> known action -> Pydantic validation -> tool execution -> observation. CampusCourier adds safety around the verifier and finish paths: ground-

ing happens before Z3, repeated UNSAT payloads are blocked, and `finish` requires the latest SAT assignment and corresponding routes. So the original lab loop is the control skeleton; the safety gates are the CampusCourier-specific additions.

```
for iteration in range(max_iterations):
    raw_response = llm.step(messages)
    messages.append({"role": "assistant", "content": raw_response})

    # TODO 1: parse raw_response as JSON (json.loads). On
    # json.JSONDecodeError, append an observation message (role
    # "user") explaining the response must be valid JSON matching
    # the required schema, and `continue`.
    try:
        step = json.loads(raw_response)
    except json.JSONDecodeError:
        messages.append({"role": "user", "content": "Observation: The response must be valid JSON match
        continue

    # TODO 2: check step["action"] is a key in TOOL_ARG_MODELS (this
    # dict already includes "finish"). If not, append an observation
    # listing the valid action names (list(TOOL_ARG_MODELS.keys())),
    # and `continue`.
    if step.get("action") not in TOOL_ARG_MODELS:
        messages.append({"role": "user", "content": f"Observation: Invalid action '{step.get('
        continue

    # TODO 3: look up model_cls = TOOL_ARG_MODELS[step["action"]] and
    # validate step.get("action_input", {}) against it:
    #     validated = model_cls(**action_input)
    # On pydantic.ValidationError, append str(e) as the observation
    # and `continue` -- do NOT proceed to call anything.
    model_cls = TOOL_ARG_MODELS[step["action"]]
    try:
        validated = model_cls(**step.get("action_input", {}))
    except ValidationError as e:
        messages.append({"role": "user", "content": f"Observation: {e}"})
        continue

    # TODO 4: if step["action"] == "finish": return
    # (validated.final_answer, messages).

    if step["action"] == "finish":
        return validated.final_answer, messages

    # TODO 5: otherwise, call tools[step["action"]](**validated.model_dump()),
    # append the JSON-stringified result as the next observation
    # message (role "user", content something like f"Observation: {json.dumps(result)}"),
    # and continue the loop.

    result = tools[step["action"]](**validated.model_dump())
    messages.append({"role": "user", "content": f"Observation: {json.dumps(result)}"})

    #raise NotImplementedError("Implement the ReAct loop body (see TODOs above)")

raise RuntimeError(f"Exceeded max_iterations={max_iterations} without calling finish.")
```

Prompt: I noticed `observe()` is doing more than just printing a result. Can you show me exactly what it changes for the trace, the UI, and the next LLM turn?

Response: `observe()` centralises what happens after every tool call. It adds the observation and a status flag to the current trace entry, appends that entry to the trace list, calls `on_step()` so the Streamlit UI can render the step immediately, and then appends the observation as a new message for the next model turn. That creates the actual ReAct loop: action -> tool execution -> observation -> new model input. Printing would only help a human watching the terminal. Appending the observation to `messages` is what allows the next LLM decision to react to the tool result.

```
def observe(entry, observation, flag="ok"):
    entry.update(observation=observation, flag=flag)
    trace.append(entry)
    if on_step:
        on_step(entry)
    messages.append({
        "role": "user",
        "content": f"Observation: {json.dumps(observation)}"
    })
```

Prompt: I have several tool actions now. Can you explain why some use `NoArgs` while `verify`, `route` and `finish` need their own Pydantic argument models?

Response: The three read-only information actions need no parameters, so they can share `NoArgs`. Verification needs typed package and robot specifications, route planning needs a robot ID and destination, and finish needs a final answer plus an optional destination and assignment mapping. Separate models make those contracts explicit and prevent one action from accidentally accepting another action's fields.

```
# -----
# Tool argument schemas (validated before any tool runs, as in the ReAct lab)
# -----
```

```
class NoArgs(BaseModel):
    pass
```

```
class VerifyArgs(BaseModel):
    packages: List[PackageSpec]
    agvs: List[AGVSpec]
```

```
class RouteArgs(BaseModel):
    robot_id: str
    destination: str
```

```
class FinishArgs(BaseModel):
    final_answer: str
    destination: Optional[str] = None
    assignments: Dict[str, str] = {}
```

```
ARG_MODELS = {
    "get_fleet_status": NoArgs,
    "get_inventory": NoArgs,
    "get_locations": NoArgs,
    "verify_dispatch_smt": VerifyArgs,
    "plan_route": RouteArgs,
    "finish": FinishArgs,
```

```
}
```

Prompt: I noticed the scripted engine can use a callable step instead of only fixed JSON. Why did I make it work that way, and how does that help the overload test?

Response: A fixed scripted step imitates a model decision, but a callable step can inspect the latest tool observation and produce its next decision from it. In the overload case the callable reads the actual Z3 observation and copies the verified assignment into the final `finish` payload. This makes the replay deterministic while still reacting to the real solver output.

```
class ScriptedLLM:
    """Offline replay (same idea as the ReAct lab's scripted_llm.py). A step may be a
    callable taking the last observation, so it can copy Z3's assignment."""
    label = "Offline scripted replay (no LLM)"

    def __init__(self, steps):
        self.steps, self.i = steps, 0

    def step(self, messages):
        step = self.steps[self.i]
        self.i += 1
        if callable(step):
            step = step(json.loads(messages[-1]["content"].removeprefix("Observation: ")))
        return json.dumps(step)
```

Prompt: I want to see how one tool result travels through the whole system. How does `observe()` end up both feeding the next LLM turn and appearing immediately in Streamlit?

Response: `observe()` adds the observation and status flag to the trace, calls `on_step(entry)`, and appends the observation to the next model message. In `app.py`, `on_step()` appends the entry to the local `steps` list, clears the placeholder, and renders the updated trace. The same event therefore feeds both the next model turn and the human-facing UI.

```
def on_step(entry):
    steps.append(entry)
    working.empty()
    stream.markdown(step_html(len(steps), entry), unsafe_allow_html=True)

try:
    if mode.startswith("OpenAI"):
        llm = agent.OpenAILLM()
    elif order == PRESETS["Low-battery trap"]:
        llm = agent.scripted_low_battery()
    else:
        llm = agent.scripted_delivery(overloaded=order == PRESETS["Overload"])
    run = agent.run_agent(llm, order, on_step=on_step)
```

Prompt: After a run, the Streamlit app calls `st.rerun()`. What state is actually surviving that rerun, and what state is coming from SQLite instead?

Response: Streamlit reruns the script from the top after the dispatch finishes. The completed result is saved into `st.session_state.result`, so the next execution can render it again. The operational fleet and inventory remain persisted in SQLite rather than relying on Streamlit session memory.

```
try:
    if mode.startswith("OpenAI"):
        llm = agent.OpenAILLM()
    elif order == PRESETS["Low-battery trap"]:
        llm = agent.scripted_low_battery()
    else:
        llm = agent.scripted_delivery(overloaded=order == PRESETS["Overload"])
```

```

        run = agent.run_agent(llm, order, on_step=on_step)
    except Exception as exc: # network down, bad API key, replay out of steps: never crash mid-demo
        hint = ("Check the wifi and OPENAI_API_KEY, or switch the sidebar to Offline scripted replay."
                if mode.startswith("OpenAI") else "Offline replay only covers the three preset orders.")
        run = {"ok": False, "final_answer": f"The reasoning engine stopped ({type(exc).__name__}). {hint}",
              "destination": None, "assignments": {}, "packages": [], "routes": {}, "trace": steps}
    if run["ok"]:
        memory.record_dispatch(order, run["destination"], run["packages"], run["assignments"], run["routes"])
        wandb_log.log_run(order, run, engine="live" if mode.startswith("OpenAI") else "offline")
        st.session_state.result = run | {"order": order}
        st.rerun()
    else:

```

Prompt: I added exception handling around the live run. Can you explain what it preserves when the model, network, API key or scripted replay fails instead of just hiding the error?

Response: The exception is converted into a structured failed run with `ok=False`, an explanatory final message, empty assignments and packages, and the trace collected so far. The UI can therefore explain the failure and suggest checking the API key/network or switching to offline replay. It does not turn an exception into a successful dispatch.

```

    try:
        if mode.startswith("OpenAI"):
            llm = agent.OpenAILLM()
        elif order == PRESETS["Low-battery trap"]:
            llm = agent.scripted_low_battery()
        else:
            llm = agent.scripted_delivery(overloaded=order == PRESETS["Overload"])
        run = agent.run_agent(llm, order, on_step=on_step)
    except Exception as exc: # network down, bad API key, replay out of steps: never crash mid-demo
        hint = ("Check the wifi and OPENAI_API_KEY, or switch the sidebar to Offline scripted replay."
                if mode.startswith("OpenAI") else "Offline replay only covers the three preset orders.")
        run = {"ok": False, "final_answer": f"The reasoning engine stopped ({type(exc).__name__}). {hint}",
              "destination": None, "assignments": {}, "packages": [], "routes": {}, "trace": steps}
    if run["ok"]:
        memory.record_dispatch(order, run["destination"], run["packages"], run["assignments"], run["routes"])
        wandb_log.log_run(order, run, engine="live" if mode.startswith("OpenAI") else "offline")
        st.session_state.result = run | {"order": order}
        st.rerun()
    else:

```

Prompt: I used an ASCII map in `world.py` rather than starting with a raw 0/1 grid. Why is that useful, and how do the same characters become room coordinates and hazard cells?

Response: The string makes the map easy to inspect and edit. `ROWS` splits it into fixed-width rows, and `cells(char)` searches every coordinate for a particular marker. The same helper creates the room coordinates from B, X, P, and S, and creates the wet and quiet-cell sets from ~ and g. That means the named locations are derived from the map rather than separately hard-coded.

```

CAMPUS_MAP = """
#####
#B...#.....#.....#
#.#.#.#####.#####
#.#...~~~.....#..P.#.#
#.#.###.##.###.###.###
#...#...#...#...#...#
###.#.###.#.#####.###
#.....#...#...#...#
#..###.#####S#####
#...#.....gggg.....#
#.#.#####.gggg#####

```

```

#.#.....#.....X#
#####
"""

ROWS = CAMPUS_MAP.strip().splitlines()
HEIGHT, WIDTH = len(ROWS), len(ROWS[0])

LOCATION_CODES = {"B": "Supply Room", "X": "Studio 3", "P": "Library", "S": "Corridor B"}

def cells(char):
    return {(x, y) for y, row in enumerate(ROWS) for x, c in enumerate(row) if c == char}

LOCATIONS = {name: next(iter(cells(code))) for code, name in LOCATION_CODES.items()}
WET = cells("~")
GAS = cells("g")

```

Prompt: I have `world.plan_route()` wrapped around the old planner now. What extra information is it adding around the A* result, and why do I need more than just the path?

Response: It adds a project-level result around the lab planner: `reachable`, the cell list, number of steps, A* nodes expanded, planning time, and counts of wet and quiet cells on the route. `path` is what the planner needs, while the other values let the UI, tests, and debugging code explain and compare routes without modifying the A* implementation itself.

```

def plan_route(start, goal, quiet_wheels=True):
    env = make_nav_env(avoid_quiet=not quiet_wheels)
    path, expanded, seconds = ForkliftPlanner(env).a_star(tuple(start), tuple(goal), "manhattan")
    return {
        "reachable": bool(path),
        "path": [list(p) for p in path],
        "steps": max(len(path) - 1, 0),
        "nodes_expanded": expanded,
        "wet_cells_on_route": sum(p in WET for p in path),
        "quiet_cells_on_route": sum(p in GAS for p in path),
        "quiet_zone": "allowed (quiet-wheeled)" if quiet_wheels else "avoided (not quiet-wheeled)",
    }

```

Prompt: Can you explain the Node dataclass in my A* code? I want to understand what `g`, `h`, `f` and `parent` are doing and why the comparisons and hash are implemented there.

Response: The Node object carries both the location and the search bookkeeping. `g` is the cost from the start, `h` is the heuristic estimate to the goal, and `f = g + h` is what A* uses to prioritise the open set. `parent` lets the planner reconstruct the final route when it reaches the goal. `__lt__` returns `self.f < other.f`, which is what lets `heapq` order Nodes by the lowest f-score. `__eq__` compares only `(x, y)`, because two Nodes representing the same cell are logically the same grid state. `__hash__` hashes `(x, y)` so Nodes can be stored in a set. So this class is not the path planner itself; it is the search-state record that the planner keeps updating.

```

@dataclass
class Node:
    x: int
    y: int
    g: float = float('inf')
    h: float = float('inf')
    f: float = float('inf')
    parent: 'Node' = None

    def __lt__(self, other):
        return self.f < other.f

```

```

def __eq__(self, other):
    if not isinstance(other, Node):
        return False
    return self.x == other.x and self.y == other.y

def __hash__(self):
    return hash((self.x, self.y))

```

Prompt: I am always getting confused by x/y versus row/column. Can you check how `is_valid_coord()`, `is_valid()` and `grid[y][x]` work together and what would break if I mixed the indices up?

Response: `is_valid_coord()` checks the array boundaries first, so an invalid coordinate cannot be indexed. `is_valid()` then adds the second condition: the cell value must be 0, which is the project's representation of free space. The important convention is that the function receives (x, y) but the Python list is accessed as `grid[y][x]`: row first, then column. If I swapped that to `grid[x][y]`, the planner would interpret rows as columns and could either read the wrong cells or go out of bounds on a non-square map. The campus adapter preserves this convention, which is why the A* code itself does not need a campus-specific rewrite.

```

def is_valid_coord(self, x: int, y: int) -> bool:
    return 0 <= x < self.width and 0 <= y < self.height

def is_valid(self, x: int, y: int) -> bool:
    return self.is_valid_coord(x, y) and self.grid[y][x] == 0

```

Prompt: Can you walk me through one A* expansion using my actual planner? I want to understand the four neighbours, `tentative_g`, and how the heuristic fits the movement model.

Response: `get_neighbors()` tries exactly four direction vectors: (0, 1), (1, 0), (0, -1), and (-1, 0). It keeps only the coordinates for which `is_valid()` is true, so walls and out-of-bounds cells disappear before the planner sees them. Because every legal movement costs one, A* increments `g` with `current.g + 1`. That is why Manhattan distance is a natural admissible heuristic for this grid. There are no diagonal actions in the environment, so allowing a diagonal-aware movement assumption would describe a different problem.

```

def get_neighbors(self, x: int, y: int):
    directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
    return [
        (x + dx, y + dy)
        for dx, dy in directions
        if self.is_valid(x + dx, y + dy)
    ]

```

```
tentative_g = current.g + 1
```

Prompt: What happens when A* reaches the same cell by two different routes? I want to understand what `open_dict` stores and why a cheaper `tentative_g` changes the parent.

Response: `open_dict` keeps the best known Node for each coordinate. When a neighbour is discovered, the planner computes `tentative_g`. If the cell is new, or the new route is cheaper than the stored `g`, the existing Node is reused or a new one is created, its `parent` becomes the current node, its `g` is lowered, then `h` and `f` are recalculated and it is pushed onto the heap. The parent change matters because the best route to the goal is reconstructed by following those parent pointers backwards. A* is therefore maintaining both the cost-to-reach information and the chain needed to recover the actual route, not just the distance number.

```
tentative_g = current.g + 1
```

```

if (x, y) not in open_dict or (tentative_g < open_dict[(x, y)].g):
    if (x, y) in open_dict:
        neighbor = open_dict[(x, y)]
    neighbor.parent = current
    neighbor.g = tentative_g

```

```

neighbor.f = neighbor.g + neighbor.h
heapq.heappush(open_set, neighbor)
open_dict[(x, y)] = neighbor

```

Prompt: I spotted something suspicious in the reused A* code: `closed_set` stores Nodes, but the membership test uses `(x, y)`. Does that really work, and does it affect the routes I am getting?

Response: Strictly speaking, the types do not match. `closed_set.add(current)` stores Nodes, while `(x, y)` in `closed_set` tests membership using a tuple. Because a tuple is not equal to a Node, that membership test does not actually skip already-closed cells in the way the comment intends. The reason the project can still behave correctly on these small grids is that `open_dict` controls the better-g-score updates and the search eventually exhausts the relevant states. The issue is still worth acknowledging because it is a quality problem in the inherited lab implementation. A cleaner implementation would make the closed set contain coordinate tuples, or consistently compare Node objects. I would not silently claim that this part of the lab implementation is perfect just because the project tests pass.

```

closed_set: Set[Node] = set()
...
closed_set.add(current)

for x, y in self.env.get_neighbors(current.x, current.y):
    if (x, y) in closed_set:
        continue

```

Prompt: I want to make sure I actually understand the heuristics I am using. What do Manhattan, Euclidean and Chebyshev calculate here, and why is Manhattan the natural choice for this map?

Response: Manhattan returns $\text{abs}(x_1 - x_2) + \text{abs}(y_1 - y_2)$, which is the number of horizontal and vertical moves needed if there were no walls. Euclidean is the straight-line distance, and Chebyshev is the maximum coordinate difference, which corresponds naturally to an eight-way movement model. For this project, movement is four-way and each step costs one, so Manhattan never overestimates the true shortest path. It is therefore admissible for the search problem. Euclidean is also a lower bound here, but it is less aligned with the actual movement metric. Chebyshev is mainly the natural metric when diagonal moves cost one, which CampusCourier does not allow.

```

def manhattan_distance(self, x1, y1, x2, y2):
    return abs(x1 - x2) + abs(y1 - y2)

def chebyshev_distance(self, x1, y1, x2, y2):
    return max(abs(x1 - x2), abs(y1 - y2))

def euclidean_distance(self, x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

```

Prompt: I could have written a new shortest-path algorithm or let the LLM suggest moves, so why did I keep the Lab 1 A* implementation and build a campus adapter around it instead?

Response: The reused planner already expects a grid-like environment with `get_neighbors()` and heuristic functions. That interface is enough for CampusCourier. I did not need to rewrite the search algorithm; I only had to create an environment whose geometry represents the campus. The adapter in `world.py` therefore owns the new campus map, converts `#` into obstacles, optionally masks the quiet zone, and calls `ForkliftPlanner(...).a_star(..., "manhattan")`. The planner still performs the actual open-set search, parent tracking, g-score updates, and path reconstruction. This is exactly the kind of reuse the course guideline permits: keep the working algorithm and adapt the inputs around it instead of regenerating the algorithm with an LLM.

```

import heapq
import time
from typing import List, Tuple, Dict, Set
from models import Node
from environment import SurgicalLabEnvironment

class ForkliftPlanner:
    """A* path planner comparing different heuristics"""

```

```

def __init__(self, environment: SurgicalLabEnvironment):
    self.env = environment

def a_star(self, start: Tuple[int, int], goal: Tuple[int, int], heuristic_type: str = 'manhattan') -> T:
    start_time = time.time()
    nodes_expanded = 0

    # -----
    # SETUP: We have initialized the starting node for you!
    # -----
    start_node = Node(start[0], start[1])
    start_node.g = 0

    # Choose the correct heuristic based on the function argument
    if heuristic_type == 'manhattan':
        start_node.h = self.env.manhattan_distance(start[0], start[1], goal[0], goal[1])
    elif heuristic_type == 'euclidean':
        start_node.h = self.env.euclidean_distance(start[0], start[1], goal[0], goal[1])
    elif heuristic_type == 'chebyshev':
        start_node.h = self.env.chebyshev_distance(start[0], start[1], goal[0], goal[1])
    else:
        raise ValueError("Invalid heuristic")

    start_node.f = start_node.g + start_node.h

    # Data Structures you will need
    open_set = []
    heapq.heappush(open_set, start_node)

    closed_set: Set[Node] = set()
    open_dict: Dict[Tuple[int, int], Node] = {(start[0], start[1]): start_node}

    path = []
    # -----
    # TODO 1: THE CORE LOOP
    # Loop as long as there are nodes in the open_set
    # -----

    while open_set:
        # 1. Pop the node with the lowest f-score from open_set using heapq
        current = heapq.heappop(open_set)

        # 2. (Optional but recommended) Check if this node is stale using open_dict

        # 3. Check if the current node is the goal!
        # If yes, reconstruct the path using the .parent attributes, reverse it, and return it.
        if (current.x, current.y) == goal:
            while current:
                path.append((current.x, current.y))
                current = current.parent
            path.reverse() # Flips list to Start -> Goal
            return path, nodes_expanded, time.time() - start_time

        # 4. Add the current node to the closed_set and increment nodes_expanded
        closed_set.add(current)

```

```

nodes_expanded += 1

# -----
# TODO 2: EVALUATING NEIGHBORS
# -----
# 5. Use self.env.get_neighbors(current.x, current.y) to get valid moves

for x,y in self.env.get_neighbors(current.x, current.y):

    if (x,y) in closed_set:
        continue

    neighbor = Node(x, y)

    tentative_g = current.g + 1

    if((x, y) not in open_dict) or (tentative_g < open_dict[(x, y)].g):
        if (x,y) in open_dict:
            neighbor = open_dict[(x, y)]
        neighbor.parent = current

```

Prompt: Can you help me unpack predict() in my Lab 2 HMM? I understand the equation, but I want to see exactly what belief_state @ T[action] is doing to the belief vector.

Response: The belief vector stores my current probability distribution over all possible hidden cells. T[action] is the transition matrix for the chosen movement action. The matrix multiplication sums the probability of arriving in each possible next state from all previous states. So the result is not saying “the robot definitely moved here.” It is the predicted distribution before seeing the new sensor reading. A cell gets more probability when several likely previous states could transition into it under that action. The old scaffold’s placeholder copied the belief unchanged. My completed version uses the actual transition model supplied by the HMM lab.

```
predicted_belief = self.belief_state @ self.T[action]
```

Prompt: I understand the Bayes update on paper, but I want to connect it to my code. Why do I multiply by the emission probabilities and then divide by Z?

Response: The predicted belief tells me how likely each hidden state was before observing the sensor. self.E[observation] gives the likelihood of seeing that observation in each hidden state. Multiplying them element-by-element therefore gives an unnormalised posterior score for every state. Z = np.sum(updated_belief) is the total probability mass. Dividing by Z makes the numbers add to one again. Without that division, the array would still reflect relative likelihoods but would no longer be a probability distribution, so the next prediction and the dashboard’s confidence value would be based on invalid probabilities.

```

updated_belief = predicted_belief * self.E[observation]
Z = np.sum(updated_belief)
updated_belief = updated_belief / Z

```

Prompt: My world code uses (x, y), while the HMM uses a flattened state index. Can you help me derive the conversion so I know exactly where the swap happens and why it is reversible?

Response: A two-dimensional cell (r, c) is flattened by counting how many full rows come before it and then adding the column: state = r * width + c. That makes every grid cell correspond to exactly one vector position. CampusCourier’s world.py uses (x, y), where x is the column and y is the row. The HMM environment expects (row, column), so perception.py deliberately calls coord_to_state(y, x). After the filter produces a state, the conversion back swaps the coordinates again before displaying the guessed (x, y) cell. Keeping these swaps inside the adapter prevents coordinate-convention bugs from leaking into the actual A* planner.

```

def state_of(cell):
    x, y = cell
    return env().coord_to_state(y, x)

```

Prompt: How does `action_between()` convert an A* move into the action numbering used by the HMM transition matrix? I want to see what would happen if two directions were accidentally swapped.

Response: `action_between()` looks only at the coordinate difference. A negative y difference is North, positive y is South, positive x is East, and otherwise it is West. Those four cases return the action indices the Lab 2 environment expects: 0, 1, 2, 3. If east and south were swapped, the filter would update with the wrong transition matrix every time the robot moved in that direction. The actual route would still be correct because A* is independent of the HMM, but the belief trace would be systematically describing the wrong motion model.

```
if dy < 0:
    return 0 # North
if dy > 0:
    return 1 # South
if dx > 0:
    return 2 # East
return 3    # West
```

Prompt: What is actually noisy in my perception simulation? Can you walk through how `sample_observation()` uses the emission model and what the robot is supposed to be estimating?

Response: The true cell is known to the simulator, but the observation is intentionally noisy. The emission matrix contains the probability of different wall-count readings for each hidden cell. `sample_observation()` takes the column corresponding to the true cell, normalises it so it can be used as sampling weights, and then chooses one observation. That means the HMM is not receiving the true (x, y) location. It is receiving a sensor-like measurement generated from the probability model. The filter then has to infer the location from the sequence of actions and noisy observations.

```
column = env().E[:, state_of(cell)]
total = column.sum()
if total == 0:
    return 0
return int(rng.choices(range(len(column)), weights=column / total)[0])
```

Prompt: The robot starts from the Supply Room, so why did I initialise the belief on the known start cell instead of using a uniform prior? What changes if I make it uniform?

Response: The project actually knows where the robot starts: the database contains its starting (x, y), and the perception layer builds the initial belief around the dock cell. That gives the filter a strong prior rather than pretending the robot could have appeared anywhere on the map. A uniform prior would be valid if the starting location were genuinely unknown, but it makes the first few observations do much more work. With a known dock start, the filter mainly tracks uncertainty caused by noisy movement observations rather than uncertainty over the entire map from time zero.

```
# Conceptually, the belief is initialised from the known dock state
belief = np.zeros(env().num_states)
belief[state_of(start)] = 1.0
```

Prompt: Why does the HMM use a complete rectangular state space that includes wall-cell indices? Can you show how `coord_to_state()` and `state_to_coord()` still make that indexing work?

Response: The rectangular state space gives every (row, column) pair a stable index, which makes the matrices simple and fixed-size. `coord_to_state(r, c)` uses `r * width + c`. The inverse uses integer division and modulo: `state // width` recovers the row and `state % width` recovers the column. Wall states remain represented as indices but are assigned zero probability in the model.

```
def coord_to_state(self, r: int, c: int) -> int:
    """Converts 2D grid coordinate to 1D vector index."""
    return r * self.width + c

def state_to_coord(self, state: int) -> tuple[int, int]:
    """Converts 1D vector index to 2D grid coordinate."""
    return (state // self.width, state % self.width)
```

```
def is_walkable(self, r: int, c: int) -> bool:
    if 0 <= r < self.height and 0 <= c < self.width:
        return self.grid[r, c] == 0
    return False
```

Prompt: Can you walk through one row of the transition model and show where the 0.8, 0.1 and 0.1 probabilities go when the intended move or a slip hits a wall?

Response: For each state/action pair, the model first tries the intended direction with probability 0.8. If that move is blocked, the destination becomes the current cell, so the probability is not lost. It then does the same for the two slip directions at 0.1 each. The three contributions are added into the destination state columns of T, giving a full probability distribution for the next hidden state.

```
def _build_transition_matrix(self) -> np.ndarray:
    """
    Builds Transition Tensor T[Action, From_State, To_State].
    Physical Slip Model: 80% chance intended move succeeds,
    10% chance slip right, 10% chance slip left.
    If hitting a wall, the robot stays in place.
    """
    T = np.zeros((4, self.num_states, self.num_states))
    moves = [(-1, 0), (1, 0), (0, 1), (0, -1)] # N, S, E, W
    slips = [
        [2, 3], # N (0) slips E (2), W (3)
        [2, 3], # S (1) slips E (2), W (3)
        [0, 1], # E (2) slips N (0), S (1)
        [0, 1]  # W (3) slips N (0), S (1)
    ]

    for s in range(self.num_states):
        r, c = self.state_to_coord(s)
        if not self.is_walkable(r, c):
            continue # Wall states stay zero probability

        for a in range(4):
            # 1. Intended direction (80% probability)
            dr, dc = moves[a]
            nr, nc = (r + dr, c + dc) if self.is_walkable(r + dr, c + dc) else (r, c)
            T[a, s, self.coord_to_state(nr, nc)] += 0.8

            # 2. Slip directions (10% probability each)
            for slip_a in slips[a]: # Directly iterate over the two slip action integers
                s_dr, s_dc = moves[slip_a]
                snr, snc = (r + s_dr, c + s_dc) if self.is_walkable(r + s_dr, c + s_dc) else (r, c)
                T[a, s, self.coord_to_state(snr, snc)] += 0.1

    return T
```

Prompt: The emission matrix is indexed as E[observation, state]. Can you explain why that orientation lines up with the update code?

Response: Each column represents one hidden state and each row represents one possible observation. So E[o, s] is the likelihood of observing wall count o when the robot is actually in state s. That is why update() can select self.E[observation]: it gets the likelihood for that observed value across every possible hidden state, which is then multiplied by the predicted belief.

```
def _build_emission_matrix(self) -> np.ndarray:
    """
    Builds Sensor Emission Matrix E[Observation, State].
```

```

Observation = Number of adjacent walls (0, 1, 2, 3, or 4).
Corrupted array emits true count with probability `sensor_accuracy`,
and nearby counts with remaining probability.
"""
num_observations = 5 # Wall counts 0 to 4
E = np.zeros((num_observations, self.num_states))
moves = [(-1, 0), (1, 0), (0, 1), (0, -1)]

for s in range(self.num_states):
    r, c = self.state_to_coord(s)
    if not self.is_walkable(r, c):
        continue

    # Count true surrounding wall obstacles
    wall_count = 0
    for dr, dc in moves:
        if not self.is_walkable(r + dr, c + dc):
            wall_count += 1

    for obs in range(num_observations):
        if obs == wall_count:
            E[obs, s] = self.sensor_accuracy
        else:
            # Uniform noisy distribution over incorrect readings
            E[obs, s] = (1.0 - self.sensor_accuracy) / (num_observations - 1)

    return E

```

Prompt: The HMM part looks impressive in the dashboard, but I want to be precise about what it really does. Is it actually changing A*'s route or correcting the robot position, or is it currently just a perception demonstration?

Response: In the current version, the HMM is an **integrated perception demonstration**, not a closed-loop controller. The route is first produced by A. *perception.walk()* then walks through that already-planned path, samples a noisy wall-count observation at each step, runs the Lab 2 forward filter, and stores the belief, most likely state, and whether the guess matches the true cell. *app.py* uses those snapshots to draw the heatmap and accuracy/confidence summary. The HMM does not currently replace the route planner's next waypoint, does not correct the database position, and does not re-run A from a filtered estimate. I would rather state that clearly than imply that the project already has a full localisation-and-control feedback loop.

```

def walk(path, seed=0):
    """
    Walk a planned route one step at a time, feeding the filter a noisy wall count
    at each step, and record what it believed.

    Returns a list of snapshots, one per step:
        true: the real (x, y)
        observation: the noisy wall count the robot got
        belief: (height, width) array of probabilities
        guess: the filter's most likely (x, y)
        correct: whether the guess matched the truth
    """
    if len(path) < 2:
        return []

    rng = random.Random(seed)
    e = env()
    estimator = HMMStateEstimator(e.num_states, e.T, e.E)

```

```

# The filter starts uniform over the whole floor, which is the right prior when a
# robot wakes up lost. Here it does not: it undocks from a known room, so the belief
# starts on that cell. Everything after this point is the lab filter doing the work,
# spreading the belief on each move and sharpening it on each wall count.
start = np.zeros(e.num_states)
start[state_of(tuple(path[0]))] = 1.0
estimator.belief_state = start

```

```

snapshots = []

```

```

for previous, current in zip(path, path[1:]):
    action = action_between(previous, current)
    observation = sample_observation(current, rng)
    belief = estimator.bayesian_filter_step(action, observation)

    guess_r, guess_c = e.state_to_coord(int(np.argmax(belief)))
    snapshots.append({
        "true": tuple(current),
        "observation": observation,
        "belief": belief.reshape(e.height, e.width).copy(),
        "guess": (guess_c, guess_r),
        "correct": (guess_c, guess_r) == tuple(current),
    })
return snapshots

```

```

def summary(snapshots):
    """Two numbers for the dashboard caption: how often it was right, and how sure it got."""
    if not snapshots:
        return {"steps": 0, "hit_rate": 0.0, "final_confidence": 0.0, "final_guess": None}
    hits = sum(s["correct"] for s in snapshots)
    near = sum(abs(s["guess"][0] - s["true"][0]) + abs(s["guess"][1] - s["true"][1]) <= 1
               for s in snapshots)

```

Prompt: I tried an earlier UI prototype before the final dashboard. Can you explain what role that prototype played and how the final Streamlit app ended up connected to the real backend rather than being a mock-up?

Response: The Tkinter prototype was an earlier design experiment for the interface. It helped me settle the visual hierarchy: fleet state, reasoning trace, verdict, and a clear distinction between “built” and future work. The final product is the Streamlit dashboard in `app.py`. The final UI is integrated with the real execution path: `run_agent()` streams trace entries through `on_step`, the current verdict is derived from the actual trace, routes are taken from the real run result, and the HMM panel consumes the real `perception.walk()` output. The interface is therefore not just a screenshot or scripted front end. This is also one of the areas where LLM assistance was explicitly appropriate under the course guidelines: layout, UI wording, component organisation, and presentation were part of the integration/design work rather than the essential lab algorithm implementation.

```

        steps.append(entry)
        working.empty()
        stream.markdown(step_html(len(steps), entry), unsafe_allow_html=True)

    try:
        if mode.startswith("OpenAI"):
            llm = agent.OpenAILLM()
        elif order == PRESETS["Low-battery trap"]:
            llm = agent.scripted_low_battery()
        else:
            llm = agent.scripted_delivery(overloaded=order == PRESETS["Overload"])
        run = agent.run_agent(llm, order, on_step=on_step)

```

```

except Exception as exc: # network down, bad API key, replay out of steps: never crash mid-demo
    hint = ("Check the wifi and OPENAI_API_KEY, or switch the sidebar to Offline scripted replay."
            if mode.startswith("OpenAI") else "Offline replay only covers the three preset orders.")
    run = {"ok": False, "final_answer": f"The reasoning engine stopped ({type(exc).__name__}). {hint}"
           "destination": None, "assignments": {}, "packages": [], "routes": {}, "trace": steps}
if run["ok"]:
    memory.record_dispatch(order, run["destination"], run["packages"], run["assignments"], run["rou
wandb_log.log_run(order, run, engine="live" if mode.startswith("OpenAI") else "offline")
st.session_state.result = run | {"order": order}
st.rerun()
else:

```

Prompt: Now that the pieces exist, can you walk me through one normal order from the moment I type it into Streamlit to the point where the database is updated? I want to understand the whole call chain before I start testing edge cases.

Response: I would start at `app.py`. The chat input produces an `order` string and the UI chooses either the live `OpenAILLM` client or one of the offline scripted stories. The important call is `agent.run_agent(llm, order, on_step=on_step)`. The callback is there so each tool step can immediately appear in the Streamlit reasoning panel. Inside `run_agent()`, the order becomes the first message. The coordinator then repeatedly asks the LLM for one JSON step. The possible actions are deliberately small: read fleet state, read inventory, read locations, plan a route, call the SMT verifier, or finish. The practical sequence for a normal order is roughly: read the fleet -> read inventory -> decide on a package list and robot set -> run the grounding check -> call Z3 -> plan the routes for the robots in the returned assignment -> finish. The `finish` action is not trusted just because the model says “done”; it is checked against the last SAT result and the route records. Only after `run_agent()` returns `ok=True` does `app.py` call `memory.record_dispatch(...)`. That means the UI is not itself deciding anything. It is presenting the state and streaming the result of a deterministic execution path.

```

steps.append(entry)
working.empty()
stream.markdown(step_html(len(steps), entry), unsafe_allow_html=True)

try:
    if mode.startswith("OpenAI"):
        llm = agent.OpenAILLM()
    elif order == PRESETS["Low-battery trap"]:
        llm = agent.scripted_low_battery()
    else:
        llm = agent.scripted_delivery(overloaded=order == PRESETS["Overload"])
    run = agent.run_agent(llm, order, on_step=on_step)
except Exception as exc: # network down, bad API key, replay out of steps: never crash mid-demo
    hint = ("Check the wifi and OPENAI_API_KEY, or switch the sidebar to Offline scripted replay."
            if mode.startswith("OpenAI") else "Offline replay only covers the three preset orders.")
    run = {"ok": False, "final_answer": f"The reasoning engine stopped ({type(exc).__name__}). {hint}"
           "destination": None, "assignments": {}, "packages": [], "routes": {}, "trace": steps}
if run["ok"]:
    memory.record_dispatch(order, run["destination"], run["packages"], run["assignments"], run["rou
wandb_log.log_run(order, run, engine="live" if mode.startswith("OpenAI") else "offline")
st.session_state.result = run | {"order": order}
st.rerun()
else:

```

Prompt: I do not want to test only the happy path. Why did I choose cases like a fake Pixel battery, repeated UNSAT, low battery and the quiet-zone route, and what failure is each one trying to catch?

Response: A normal success case only proves the happy path works. The important safety claims in this project are about **rejection behaviour**, so the tests need malicious or impossible inputs. The invented-battery test checks that the model cannot bypass the database by changing a fact in the verifier payload. The repeated-UNSAT test checks that the logical loop actually forces a new plan rather than repeatedly asking Z3 the same question. The low-battery

test checks the hard exclusion rule. The quiet-zone test checks that route planning respects robot capability rather than allowing a generic shortest path through the same cells for every robot. The test file therefore mirrors the architecture: syntax, grounding, mathematical feasibility, route restrictions, and final commitment are all separate things that can fail.

```
def flags(run):
    return [t["flag"] for t in run["trace"]]

def test_routine_dispatch():
    memory.reset()
    run = agent.run_agent(agent.scripted_delivery(), "Get 2 projectors and a lab kit to Studio 3.")
    assert run["ok"], run["final_answer"]
    assert flags(run)[-2:] == ["sat", "finish"]
    assert all(run["routes"][r]["reachable"] for r in set(run["assignments"].values()))

def test_overload_is_rejected_then_relaxed():
    memory.reset()
    run = agent.run_agent(agent.scripted_delivery(overloaded=True), "Send 3 projectors and the whiteboard.")
    assert run["ok"]
    assert flags(run).index("unsat") < flags(run).index("sat")
    assert not any(p.startswith("whiteboard") for p in run["assignments"])

def test_low_battery_robot_is_refused():
    memory.reset()
    run = agent.run_agent(agent.scripted_low_battery(), "Use Pixel to carry a lab kit to the Library.")
    assert not run["ok"] and "unsat" in flags(run)

def test_invented_numbers_and_unverified_finish_are_blocked():
    memory.reset()
    liar = agent.ScriptedLLM([
        {"thought": "", "action": "verify_dispatch_smt", "action_input": {
            "packages": [{"id": "projector#1", "weight": 12}],
            "agvs": [{"id": "CB-3", "capacity": 10, "battery_pct": 95, "min_battery": 20, "drain_rate": 0.3}
        }},
        {"thought": "", "action": "finish", "action_input": {
            "final_answer": "done", "destination": "Studio 3", "assignments": {"projector#1": "CB-3"}},
        {"thought": "", "action": "finish", "action_input": {"final_answer": "cannot"}},
    ])
    run = agent.run_agent(liar, "test")
    assert flags(run) == ["blocked", "blocked", "finish"] and not run["ok"]
    assert run["trace"][0]["observation"]["status"] == "REJECTED_UNGROUNDED"

def test_repeating_an_unsat_plan_is_blocked():
    memory.reset()
    fleet = [{"id": r["robot_id"], "capacity": r["capacity"], "battery_pct": r["battery_pct"],
        "min_battery": r["min_battery"], "drain_rate": r["drain_rate"]} for r in memory.robots()]
    too_heavy = {"packages": [{"id": f"projector#{i}", "weight": 12} for i in range(1, 6)], "agvs": fleet}
    step = {"thought": "", "action": "verify_dispatch_smt", "action_input": too_heavy}
    run = agent.run_agent(agent.ScriptedLLM([step, step, {"thought": "", "action": "finish",
        "action_input": {"final_answer": "no"}}]), "test")
    assert flags(run)[:2] == ["unsat", "blocked"]
    assert run["trace"][1]["observation"]["status"] == "BLOCKED_REPEAT"
```

```

def test_battery_drain_blocks_a_load_that_capacity_allows():
    memory.reset()
    run = agent.run_agent(agent.scripted_delivery(overloaded=True), "Send 3 projectors and the whiteboard.")
    memory.record_dispatch("setup", run["destination"], run["packages"], run["assignments"], run["routes"])
    badger = next(r for r in memory.robots() if r["robot_id"] == "CB-2") # drained by the first dispatch
    spec = {k: badger[k] for k in ("capacity", "battery_pct", "min_battery", "drain_rate")} | {"id": "CB-2"}
    tank = [{"id": "projector#1", "weight": 12}]
    assert 12 <= badger["capacity"] and badger["battery_pct"] - badger["drain_rate"] * 12 < badger["min_battery"]
    assert agent.verify_dispatch_smt({"packages": tank, "agvs": [spec]}["status"] == "UNSATISFIABLE"

def test_robots_without_silent_wheels_avoid_the_quiet_zone():
    memory.reset()
    badger = agent.plan_route_tool("CB-2", "Corridor B") # not quiet-wheeled
    mole = agent.plan_route_tool("CB-1", "Corridor B") # quiet-wheeled
    assert badger["reachable"] and badger["quiet_cells_on_route"] == 0
    assert mole["quiet_cells_on_route"] > 0 and badger["steps"] > mole["steps"]
    for a in world.LOCATIONS.values():
        for b in world.LOCATIONS.values():
            assert world.plan_route(a, b, quiet_wheels=False)["quiet_cells_on_route"] == 0

if __name__ == "__main__":
    tests = [f for name, f in dict(globals()).items() if name.startswith("test_")]
    for t in tests:
        t()
        print("PASS", t.__name__)
    print(f"\nAll {len(tests)} checks passed.")

```

Prompt: Can you walk me through the battery test where the load still fits Comet's capacity but its battery after the previous dispatch is too low? I want to make sure I understand why that is a different test from capacity.

Response: Yes. The test first performs the overload scenario and commits it, which changes the database state. It then looks up Comet's **current** battery from SQLite rather than using its original seed value. The test constructs a one-projector payload and checks two different facts: $12 \leq \text{badger}["\text{capacity}"]$ is true, so capacity alone would allow the package, but $\text{badger}["\text{battery_pct}"] - \text{badger}["\text{drain_rate}] * 12 < \text{badger}["\text{min_battery}"]$ is also true. The final assertion requires Z3 to return UNSAT. That is important because it demonstrates that battery drain can reject a load that fits physically. The decision depends on the robot's current battery state after earlier work, which is exactly why persistent state matters.

```

assert run["trace"][0]["observation"]["status"] == "REJECTED_UNGROUNDED"

```

```

def test_repeating_an_unsat_plan_is_blocked():
    memory.reset()
    fleet = [{"id": r["robot_id"], "capacity": r["capacity"], "battery_pct": r["battery_pct"],
              "min_battery": r["min_battery"], "drain_rate": r["drain_rate"]} for r in memory.robots()]
    too_heavy = {"packages": [{"id": f"projector#{i}", "weight": 12} for i in range(1, 6)], "agvs": fleet}
    step = {"thought": "", "action": "verify_dispatch_smt", "action_input": too_heavy}
    run = agent.run_agent(agent.ScriptedLLM([step, step, {"thought": "", "action": "finish",
                                                         "action_input": {"final_answer": "no"}}]), "test")

    assert flags(run)[:2] == ["unsat", "blocked"]
    assert run["trace"][1]["observation"]["status"] == "BLOCKED_REPEAT"

```

```

def test_battery_drain_blocks_a_load_that_capacity_allows():
    memory.reset()
    run = agent.run_agent(agent.scripted_delivery(overloaded=True), "Send 3 projectors and the whiteboard.")

```

```
memory.record_dispatch("setup", run["destination"], run["packages"], run["assignments"], run["routes"])
badger = next(r for r in memory.robots() if r["robot_id"] == "CB-2") # drained by the first dispatch
spec = {k: badger[k] for k in ("capacity", "battery_pct", "min_battery", "drain_rate")} | {"id": "CB-2"}
tank = [{"id": "projector#1", "weight": 12}]
```

Prompt: Can you give me a manual Python-shell sequence I can use to debug one complete overload run? I want to be able to reproduce the path from reset, through grounding and Z3, through A*, and finally into the database commit.

Response: I would start with `memory.reset()`, then construct a package/robot payload from `memory.supplies()` and `memory.robots()`. I would call `grounding_problems(payload)` first, then `verify_dispatch_smt(payload)`. If that returns UNSAT, I would change the package list, call the verifier again, then call `plan_route_tool()` for each robot in the SAT assignment. Only after the finish conditions match would I call `memory.record_dispatch()`.

```
memory.reset()
payload = ...
assert grounding_problems(payload) == []
verdict = verify_dispatch_smt(payload)
# change payload after UNSAT
verdict2 = verify_dispatch_smt(new_payload)
route = agent.plan_route_tool(robot_id, "Studio 3")
memory.record_dispatch(order, destination, packages, assignments, routes)
```

Prompt: I know I have coursework around value iteration, Q-learning and conflict resolution, but I am not importing those into the main path yet. Can you help me separate what is actually running from what is still a future layer?

Response: I would show the status explicitly. Lab 3 value iteration and the Q-learning coursework were used as references for a planned adaptive layer, but the current dispatch path does not import their policy engines. They are therefore **planned/not integrated**, not hidden implementation. The final working path is the LLM coordinator + SQLite memory + grounding gate + SMT verifier + A* route planning + commit gate, with the Lab 2 HMM added as an integrated perception visualisation. The presentation files and proposal may discuss future adaptive components, but the current README and code identify those components as planned. I think this distinction is important because a technical review should describe what a user can actually run today, not what the roadmap says might be added later.

Prompt: Before I call the project finished, can you give me the honest list of things that are weaker or incomplete compared with the story I started with?

Response: The safety proof is narrower than the headline. Two rubric layers are missing or half-built. The map does not support the story. Perception does not feed into any decision. And there is no evidence of the live model working. Grouped by what they affect. What the proof covers (Q12, Q5, Q45) • Battery drain is per kg, not per step. Route length never reaches Z3. • There is no return reserve, although the proposal lists it as hard rule 5. • The quiet zone is enforced in the map, outside the proof. • The UNSAT “reason” is a Python heuristic, not an unsat core. • Nothing checks that the plan matches the order’s items or destination. Brief coverage (Q10) • The adaptive pilot (MDP, Q-learning) is not built: one whole layer. • The Corridor B conflict and self-play are not built, and the map has no real bottleneck for them. • Deadlines and time are mentioned in the orders but not modelled. World model (Q1, Q37, Q34) • “Corridor B” is a dead-end cell; Studio 3 has two entrances; there is no exam hall. • Robots teleport to the destination and never return to the Supply Room for the next load. • The belief is computed after the fact and never used. The filter assumes slip everywhere, while the true path never slips. Robustness of the gates (Q27, Q28, Q43) • The repeat block can be bypassed by reordering or renaming. • Duplicate package ids desynchronise the proof from the database. • Three of the four commit-gate branches are untested. Evidence and presentation (Q9, Q40, the front page) • Offline mode silently replays the routine story for any typed order. • There is no live-model trace. All six W&B runs are offline. • The API key is in the zip. The README has a TODO. The proposal is out of date.

- Leftover names (badger, GAS, "In tunnels") hint at an earlier scenario.

What is not a limitation, and you should say so with confidence: the reused algorithms are correct and verifiably yours; the gates are well placed; the verifier fixes are right; and the tests that exist are meaningful rather than padding.

Prompt: I want one clean status check before the final audit: which pieces are live, which are reused or adapted from coursework, and which ones are still planned? I do not want to accidentally present planned work as implemented.

Response: You cut the adaptive pilot (MDP and Q-learning), Corridor B self-play, the return-trip reserve, time and deadlines, and the quiet zone as a Z3 rule. Cutting the pilot was the expensive one, because it is a full 10% rubric layer. The dashboard itself lists what is planned:

```
app.py, lines 36-44 36 LAYERS = [
37 ("LLM coordinator + Z3 safety filter", "live"), 38 ("A* routing", "live"), 39 ("SQLite memory", "live"), 40 ("Live
dashboard", "live"), 41 ("HMM localisation (Lab 2 filter)", "live"), 42 ("Adaptive pilot (MDP + RL)", "planned"),
43 ("Corridor B conflict (self-play)", "planned"), 44 ]
```

Against the rubric ("Integrating each layer will fetch 10% [60%]"):

Brief's layer State Marks at risk 1. Human interface + safety filter (LLM + Z3) built - 2. Perception (noisy sensors) built, decorative (Q34) small 3. Route finder and conflict resolver A* built, conflict not about 5% 4. Adaptive pilot (slip + learning) not built 10% 5. Memory (database) built - 6. Live dashboard built -

The proposal's reasoning ("everything else. . . can be cut back if I run short of time. I built and tested the verifier on its own first") is the right instinct: secure the must-have first. The brief says the verifier is mandatory ("without it. . . I will have to let it go"). One thing I need to flag. Where the time went is debatable. Weights & Biases logging (wandb_log.py) and the 17-check stress test were not asked for. Your Lab_3_Aryan.zip already contains a working value_iteration_lab.py, and reusing it is explicitly allowed, so the pilot could have been the lab code plus an adapter, the same pattern as perception.py. In hindsight that was a better use of a day than the W&B runs.

Component / file	Source / origin	What was reused	What changed or was built for CampusCourier
agent.py	New integration around ReAct lab pattern	Action/observation control-flow idea	Project tools, JSON/Pydantic boundary, 12-step cap, grounding, repeat-UNSAT handling, finish/commit gate, offline scripts
memory.py	New; persistent-state pattern adapted from coursework	Tool-accessible persistent state idea	Campus fleet/inventory schema, seed state, task logging, battery/position/stock updates
world.py	New campus adapter	Lab 1 environment interface	ASCII campus map, named locations, quiet-zone masking, route wrapper
perception.py	New adapter around Lab 2	Forward-filter inputs/outputs	Campus coordinate/action/sensor conversion and HMM integration
app.py	New UI/integration	None from the essential search algorithm	Streamlit input, live reasoning stream, map, fleet/status panels, HMM display, error handling
core/planner.py, environment.py, models.py	Lab 1	A* planner, grid, Node and heuristics	Reused; campus geometry is supplied through world.py
core/hmm_filter.py, hmm_environment.py	Lab 2	Forward filter and transition/emission structure	Reused/adapted at the integration boundary; the filter is run over planned campus routes

Component / file	Source / origin	What was reused	What changed or was built for CampusCourier
core/verifier.py	SMT coursework lab	Existing verifier structure and Z3 model	Task 0 $\geq \rightarrow \leq$; Task 4 battery-drain constraint; lab docstrings were removed
test_agent.py, stress_test.py	New project-specific testing	General testing approach	Seven deterministic agent checks plus seventeen stress checks, including deliberately bad model behaviour
Value iteration / Q-learning / conflict layer	Coursework/proposal reference	Concepts and lab material	Not imported into the current dispatch path; planned only

Prompt: By the time I have all these pieces connected, I want to understand who is actually in charge of what. If the LLM proposes something, which layer is allowed to reject it and which layers are only giving information back?

Response: The LLM is the proposer. It can choose which tool to call and what candidate payload to submit, but it cannot force a successful transition. Pydantic can reject the shape of the request. Grounding can reject facts that do not match SQLite. Z3 can reject an allocation that violates the encoded mathematical constraints. A* can report that a route is unreachable under the robot's navigation rules. The finish gate can reject an assignment that was not the solver's SAT model or a robot that has not been routed to the destination. SQLite is the persistent state source and the final commit target. It does not ask the LLM for permission to change numbers; the code writes it only after the earlier gates have passed. So the authority chain is intentionally asymmetric: the model proposes, deterministic code decides whether the proposal may become state.

Prompt: I want to prove to myself that the deterministic safety path does not depend on the LLM behaving nicely. If I use the deliberately bad scripted model that lies about Pixel's battery and then tries to finish, what should the trace look like?

Response: The trace should show a `verify_dispatch_smt` action being **blocked at the grounding gate**, followed by a `finish` attempt being blocked because there is no SAT result supporting the claimed assignment. The run ends without a committed dispatch. This matters because the scripted model is intentionally behaving incorrectly. I am not testing whether a well-behaved model follows my prompt; I am testing whether the deterministic layers can stop a badly behaved model. That is the practical reason I built both `test_agent.py` and `stress_test.py` around adversarial cases. The safety story should still hold when the model is wrong.

```
bad = []
for a_name, a in world.LOCATIONS.items():
    for b_name, b in world.LOCATIONS.items():
        for quiet in (True, False):
            route = world.plan_route(a, b, quiet_wheels=quiet)
            if not route["reachable"]:
                bad.append((a_name, b_name, quiet))
            if not quiet and route["quiet_cells_on_route"]:
                bad.append(("quiet zone entered", a_name, b_name))
check("all rooms reachable, quiet zone respected", not bad, str(bad[:3]))

# 3. verifier edge cases
fleet = [{"id": r["robot_id"], "capacity": r["capacity"], "battery_pct": r["battery_pct"],
          "min_battery": r["min_battery"], "drain_rate": r["drain_rate"]} for r in memory.robots()]

empty = verify_dispatch_smt({"packages": [], "agvs": fleet})
check("empty package list does not crash", empty["status"] in ("SATISFIABLE", "UNSATISFIABLE"), str(empty))

single_huge = verify_dispatch_smt({"packages": [{"id": "crate#1", "weight": 999}], "agvs": fleet})
```

```

check("one impossible package is UNSAT", single_huge["status"] == "UNSATISFIABLE")
check(" and names it as oversized",
      "crate#1" in single_huge.get("diagnostics", {}).get("oversized_packages", []))

many = verify_dispatch_smt({"packages": [{"id": f"p#{i}", "weight": 1} for i in range(40)], "agvs": fleet})
check("40 tiny packages still solvable", many["status"] == "SATISFIABLE")

no_fleet = verify_dispatch_smt({"packages": [{"id": "p#1", "weight": 1}], "agvs": []})
check("no robots at all is UNSAT", no_fleet["status"] == "UNSATISFIABLE")

flat = [dict(f, battery_pct=5) for f in fleet]
check("whole fleet flat is UNSAT",
      verify_dispatch_smt({"packages": [{"id": "p#1", "weight": 1}], "agvs": flat})["status"] == "UNSATISFIABLE")

# capacity alone would allow this load, the drain rule must stop it
tight = [{"id": "X", "capacity": 100, "battery_pct": 50, "min_battery": 20, "drain_rate": 1.0}]
check("capacity ok but drain too high is UNSAT",
      verify_dispatch_smt({"packages": [{"id": "p#1", "weight": 40}], "agvs": tight})["status"] == "UNSATISFIABLE")
check("same load within drain budget is SAT",
      verify_dispatch_smt({"packages": [{"id": "p#1", "weight": 25}], "agvs": tight})["status"] == "SATISFIABLE")

# 4. the gates, against a model that behaves badly on purpose
fresh()
idle = [{"thought": "idle", "action": "get_fleet_status", "action_input": {}}] * 12

liar = agent.ScriptedLLM([

```

Prompt: There is a small mismatch between the real-valued battery arithmetic in Z3 and the integer battery that memory.py stores after a dispatch. Does that change anything important, and how should I describe it?

Response: It is a small implementation inconsistency that I would explicitly acknowledge. The verifier uses `z3.RealVal(str(agv.drain_rate))`, so a value such as 0.5 is represented exactly in the SMT arithmetic. That is good for avoiding binary floating-point ambiguity in the constraint expression. The commit code, however, computes the new battery in Python and then applies `int(...)`. That truncates the stored battery. For the current threshold-based examples this does not automatically make a previously accepted plan unsafe, because the verifier keeps a margin above the minimum, but it means the database state can be up to just under one percentage point lower than the exact real calculation. A cleaner future version would define one battery-update rule and use the same rounding convention both in verification and persistence.

```

(order_text, destination, packages, assignments, routes):
    """Commit a verified plan: log tasks, draw down stock, drain battery, move robots."""
    weights = {p["id"]: p["weight"] for p in packages}
    fleet = {r["robot_id"]: r for r in robots()}
    goal = LOCATIONS[destination]
    with closing(_connect()) as conn, conn: # closes the file handle and commits
        for robot_id in sorted(set(assignments.values())):
            items = [pid for pid, rid in assignments.items() if rid == robot_id]
            load = sum(weights[i] for i in items)
            new_battery = int(fleet[robot_id]["battery_pct"] - fleet[robot_id]["drain_rate"] * load)
            conn.execute(
                "INSERT INTO tasks (order_text, robot_id, items, destination, route_steps, status) "
                "VALUES (?, ?, ?, ?, ?, 'dispatched')",
                (order_text, robot_id, json.dumps(items), destination, routes[robot_id]["steps"]),
            )
            conn.execute("UPDATE robots SET battery_pct = ?, x = ?, y = ? WHERE robot_id = ?",
                          (new_battery, goal[0], goal[1], robot_id))
        for pid in assignments:
            conn.execute("UPDATE supplies SET stock = stock - 1 WHERE item = ?", (pid.split("#")[0],))

```

Prompt: The project idea talks about a battery reserve for the whole trip. Looking at the current code, what does the battery rule actually guarantee, and what is still missing around route distance and the return trip?

Response: The implemented verifier guarantees a **load-dependent battery constraint** of the form `battery_pct - drain_rate * assigned_weight >= min_battery` for a robot that actually receives a load. It does not currently encode route length, energy per movement step, or an explicit return-to-Supply-Room reserve. `memory.record_dispatch()` also moves the robot's stored position directly to the destination and applies the load-based battery calculation. The A* route length is recorded in the task log, but it is not part of the SMT battery formula. So I would not claim that the current version proves round-trip energy sufficiency. That was part of the project design direction, but the working implementation is narrower. A future version would add a route-cost term to the verified model or perform a second deterministic reserve check before commit.

Prompt: I am getting to the verification part now. Can you first explain SAT versus SMT in terms of the actual variables and arithmetic in my project?

Response: SAT asks whether true/false variables can be set to make a formula true. SMT asks the same question, but the formula may also talk about numbers, sums and inequalities. My constraints are about kilograms and battery percentages, so I need SMT. SAT (Boolean satisfiability): variables are only true or false, and the formula is built from AND, OR and NOT, usually in CNF. Solvers use CDCL (conflict-driven clause learning). It is NP-complete, but modern solvers handle millions of clauses. SMT (Satisfiability Modulo Theories): SAT plus theories that give symbols meaning, such as linear integer arithmetic, real arithmetic, arrays and bit-vectors. The usual architecture, DPLL(T), runs a SAT engine over the Boolean structure, and hands the arithmetic facts it picks to a theory solver (here, a simplex-style solver for linear arithmetic). The theory solver either agrees or reports a conflict, which the SAT engine learns from. What my problem needs: • integer variables ap in $\{0, \dots, n-1\}$, P • sums of conditional terms $p \text{ If}(ap = i, wp, 0) \leq cap_i$, • a real-valued drain, $battery_i - 0.3 \cdot Li \geq min_i$. That is quantifier-free linear arithmetic over integers and reals (the theory Z3 calls QF_LIRA). Pure SAT could only handle it by bit-blasting: encoding every number as a set of Boolean bits and every addition and comparison as a circuit. That is possible, but it is huge and opaque. SMT keeps the constraints readable, as `solver.add(assigned_weight <= agv.capacity)`, which is the point of using it for safety rules a human must audit.

Prompt: When Z3 says UNSAT, what does that actually tell me that an LLM saying 'I can't do it' would not tell me?

Response: UNSAT is a proof that no assignment, out of all npackages of them, satisfies the constraints. An LLM's "can't be done" is a guess with no certificate. It might be wrong, it might differ next time, and nothing can check it.

1. It is exhaustive. Z3 does not sample a few splits. Its search covers the whole space, pruning by learned conflicts, so "no" means none.
2. It is deterministic. Same constraints, same answer. Test 5 depends on this.
3. It is sound relative to the encoding. Unless the constraints themselves are wrong (Task 0 shows what that looks like), UNSAT cannot be a false refusal.
4. It does not argue. The overload story's 54 kg against 50 kg is not an opinion that a persuasive order can override.

Its limits are just as important. The answer is only as good as (a) the constraints encoded, which is why Q12 matters, and (b) the input numbers, which is why the grounding gate exists. "UNSAT" means "impossible under these rules for these numbers". Honest check: the explanation is not from Z3. The README says "UNSAT returns diagnostics naming the constraint that failed". It does not quite. After Z3 says UNSAT, separate Python (`verifier.py:96-148`)

does arithmetic on totals and picks a story. That case analysis is sensible, and it is the lab's. But it is a heuristic explanation, not something Z3 proved. The "packing conflict" branch even says the cause "may be" the drain interaction.

```
core/verifier.py, lines 133-148
133 else:
134     feedback = (
135         f"UNSAT: no valid assignment exists even though total
weight "
136         f"({total_weight}kg) fits within total active capacity "
137         f"({total_active_capacity}kg). This is a
packing/arrangement "
138         f"conflict, not a raw capacity shortfall – try a different "
139         f"split of which packages
travel together."
140     )
141     if any(a.drain_rate > 0 for a in active_agvs):
142         feedback += (
143             "Note: one or
more active AGVs have a nonzero drain_rate, "
144             "meaning the weight they're given also costs them battery"
145             "headroom above their own min_battery – that interaction"
146             "may be part of why no split of these packages
works, even"
147             "though the raw capacity numbers look sufficient."
148         )
```

Z3 can give a real explanation: name each constraint with `solver.assert_and_track(c, "cap_CB-2")` and read

`solver.unsat_core()`, a set of named constraints that together are already impossible. That would be a genuine “the constraint that failed”, and it would be a strong, cheap upgrade, done in your integration code by re-running a tracked solver only when the answer is UNSAT.

Prompt: I found the retry formula $1 - (1 - p)^k$ in the notes. Can you explain what it means for repeated attempts here and what assumptions I need before using it?

Response: If one attempt is right with probability p , then k independent attempts all fail with probability $(1 - p)^k$, so at least one succeeds with $1 - (1 - p)^k$. It depends on independence, on a checker that never accepts a bad plan, and on a valid plan existing at all. The arithmetic: with $p = 0.6$, three attempts give $1 - 0.43 = 0.936$, and five give 0.990 . The course’s point, which your proposal makes well, is where the safety comes from. A sound checker means a wrong attempt costs only time, never a bad dispatch. So an unreliable planner behind a sound checker is a safe system, and a reliable planner behind a leaky checker is not. The assumptions, and how your system relates to each: 1. Independence. This is the weakest assumption. LLM attempts share the same prompt, the same model and the same misconceptions, so their failures are correlated. Feedback deliberately makes them dependent too, and hopefully in a good way: each retry is informed by the UNSAT diagnostics. The formula is an intuition, not a calculation you can trust numerically. 2. Distinct attempts. The repeat block (Q28) enforces this, at least textually. 3. A sound checker (no false accepts). The three gates are the attempt to guarantee this, within the limits of what is encoded (Q12). 4. A solution exists. If the order is truly impossible, as in the low-battery story where only Pixel is allowed, then $p = 0$ and no number of retries helps. The correct outcome is a refusal, and your system gives one. 5. k is bounded. It is effectively at most about 5 verifies inside the 12-step cap.

Prompt: I want to understand why I bothered putting safety in executable checks instead of relying only on the system prompt. What does the code enforce that a prompt cannot guarantee?

Response: A better prompt raises the chance that the model is right. A sound verifier makes the cost of the model being wrong zero. Only the second one is a guarantee. Look at how the project treats the same rule twice. The prompt asks:

agent.py, lines 131-137 131 Rules: 132 1. After UNSATISFIABLE, read the feedback and diagnostics, then submit a genuinely 133 different payload (e.g. fewer or lighter items), never the same one again. 134 Tell the commander what you dropped and why. Priority when cutting load: 135 projector > lab_kit > water_case > marker_box > whiteboard. 136 2. Never finish with assignments unless the latest verify_dispatch_smt was SATISFIABLE. 137 3. Before finishing, plan_route every robot that appears in the assignments.

and the code enforces:

agent.py, lines 273-279 273 if action == “finish”: 274 refusal = None

275 if args.assignments: 276 if not last_verdict or last_verdict.get(“status”) != “SATISFIABLE”: 277 refusal = “BLOCKED: no SATISFIABLE verification backs these assignments.” 278 elif args.assignments != last_verdict[“assignments”]: 279 refusal = “BLOCKED: assignments differ from the Z3-verified model.”

Rule 2 of the prompt and lines 276-277 say the same thing. The difference is that the prompt can be ignored, and the if cannot. That is the whole argument in one place: the prompt is advice, the code is the law. Three more reasons the verifier is the better investment: 1. Models change under you. OPENAI_MODEL defaults to gpt-5-mini (agent.py:147). A model upgrade can quietly change behaviour. A verifier’s behaviour only changes when you change it. 2. Arithmetic is where LLMs fail. Sums of weights and $40 - 1.0 \times 12$ are exactly the kind of thing a model gets slightly wrong while sounding certain. 3. It is testable. test_agent.py can prove the verifier rejects a lie. No test can prove a prompt will always be obeyed. One thing I need to flag. The prompt still matters, just for a different property. The verifier gives safety (nothing bad gets through). Whether the agent finds a good plan within 12 steps, its liveness, depends entirely on the model and the prompt. A perfectly safe system that refuses everything is useless. Also, the verifier is only as strong as what it encodes. Q12 lists what it does not.

Prompt: I considered simpler allocation checks before settling on Z3. Can you compare the actual problem here with if/else checks, brute force and MILP, and explain what Z3 is buying me?

Response: Z3 is not checking a plan the model wrote. It is searching for one, which is bin-packing. If/else can check a given assignment, but it cannot prove that no assignment exists without enumerating all of them. The detail people miss is in what the model actually sends. The payload is a list of packages and a list of robots, with no assignment:

agent.py, lines 122-126 122 - verify_dispatch_smt {"packages": [{"id": "projector#1", "weight": 12}, ...], 123 "agvs": [{"id": "CB-1", "capacity": 30, "battery_pct": 85, 124 "min_battery": 20, "drain_rate": 0.5}, ...]} 125 Z3 decides which robot carries which package. Copy robot fields EXACTLY from 126 get_fleet_status. One package entry per unit, ids like "lab_kit#1", "lab_kit#2".

Z3 then chooses who carries what (verifier.py:55, one integer per package). So the question for the solver is “does any assignment exist?”, not “is this assignment OK?”. That is a different problem.

- If/else is enough to check one fixed assignment: add up the weights per robot and compare. It is not enough to answer “impossible”. For that you must rule out every one of the $3n$ assignments, and if/else checks one.
- Brute-force enumeration (itertools.product) would get the same answers here. Five packages and three robots is only $3^5 = 243$ cases. Be honest about this if asked: at this scale Z3 is not necessary for speed.
- MILP or CP-SAT (OR-Tools, PuLP) would also work and could optimise as well, for example minimising the number of robots used.
- A second LLM as judge. This is the alternative your proposal rightly rejects: a checker that can accept bad plans breaks the retry argument (Q46).

Why Z3 is still the right choice: the constraints are declarative. Adding a rule is one solver.add line rather than a change to a search loop. UNSAT is a proof over the whole space. And it is what the course asks for (“Figure out how verification with an SMT will work in your context”).

Prompt: I currently enforce the quiet-zone rule by changing the routing grid. Is that enough to call it part of the safety proof, or is it actually a separate A* rule outside the SMT constraints?

Response: Masking makes an illegal route impossible to produce, which is robust, but the quiet-zone rule then sits outside the SMT proof and outside the UNSAT diagnostics. The current implementation therefore treats it as a routing rule rather than as an SMT constraint.

The masking is done in world.py:

```
def make_nav_env(avoid_quiet=False):
    env = SurgicallabEnvironment(map_type="trivial")
    env.width, env.height = WIDTH, HEIGHT
    env.grid = [[1 if c == "#" or (avoid_quiet and c == "g") else 0
                 for c in row] for row in ROWS]
    return env
```

The robot capability used to decide whether g cells should be masked comes from the database, not from the model. That means a non-quiet robot literally searches a graph in which the quiet-zone cells are blocked. A* can therefore guarantee that the route it returns does not contain those cells for that robot.

The limitation is that Z3 does not know the route. If the route is impossible for a non-quiet robot, the routing layer reports `reachable: false`; it is not an SMT UNSAT explanation. A more rigorous future version could add a deterministic post-check or an explicit route-derived SMT condition. For the current implementation, the honest statement is that the quiet-zone rule is enforced by A* masking, not proved inside Z3.

Prompt: Before I describe the verifier as a guarantee, can you challenge the strongest safety claim I make and list exactly what the current Z3 model does not prove?

Response: “Every dispatch plan must be proved safe by a Z3 solver before a robot moves.” What Z3 proves is that the model’s numbers satisfy three inequalities. Distance, the return trip, the quiet zone, item pickup and whether the plan matches the order are all outside the proof. The claim is the README’s very first sentence. Here is everything Z3 actually checks:

core/verifier.py, lines 57-81 57 # 1. Bounds: Every package mapped to a valid AGV index

```
58 for p_id, var in pkg_vars.items(): 59 solver.add(var >= 0, var < num_agvs) 60 61 # 2. Battery Theory Check:
Low battery AGVs cannot receive assignments 62 for idx, agv in enumerate(payload.agvs): 63 if agv.battery_pct
< agv.min_battery: 64 for p_id, var in pkg_vars.items(): 65 solver.add(var != idx) 66 67 # 3. Weight Theory
Check: Sum of package weights <= AGV capacity 68 for idx, agv in enumerate(payload.agvs): 69 assigned_weight =
z3.Sum([ 70 z3.If(pkg_vars[p_id] == idx, p.weight, 0) 71 for p in payload.packages 72 ]) 73 solver.add(assigned_weight
<= agv.capacity) 74 75 # 4. Battery-Drain Theory Check (Task 4 – fill in battery_drain_constraint above) 76 for
idx, agv in enumerate(payload.agvs): 77 assigned_weight = z3.Sum([ 78 z3.If(pkg_vars[p_id] == idx, p.weight, 0)
79 for p in payload.packages 80 ]) 81 solver.add(battery_drain_constraint(agv, assigned_weight))
```

Bounds, battery above minimum, capacity, and drain per kg. Now what a sceptic can show it doesn't check:

1. Distance. `battery_left = battery - drain * load` (verifier.py:39). Atlas carrying 24 kg ends at 73% whether the trip is 0 steps or 32. A robot at 21% battery carrying 1 kg passes, even with a 300-step route.
2. The return trip. The proposal lists "Return reserve" as hard rule 5. It is not in the code, and the README admits it.
3. The quiet zone. It lives in the map (Q5), not in the proof.
4. Picking up the items. Robots never go back to the Supply Room (Q37), so the physical story is broken from the second order onwards.
5. The order itself. Grounding checks that every number matches the database. Nothing checks that the packages or destination match what the human asked for. If the order says "Library" and the model plans Studio 3, every gate passes.

What I'd do: Two moves, and you should do both. Reword to something you can defend: "Every dispatch is checked by Z3 against capacity, minimum battery and load-dependent battery drain, using numbers verified against the database. Routing rules are enforced by the planner." That is still a strong claim, and it is true. Strengthen the cheap part: make drain depend on distance. The integration-side way, which leaves the lab file alone, is to compute per robot battery $\text{eff} = \text{battery} - \text{step_cost} \times \text{steps}$ from the A* route and pass that to Z3 as `battery_pct`. Grounding would then compare against the route-adjusted value, and you would document that this changes what the field means. Putting it inside `core/verifier.py` would be a third change to lab code, so disclose it if you go that way.

Prompt: Why does `verify_dispatch_smt()` validate the payload with Pydantic before it even creates the Z3 solver? What kind of bad input should be caught at that earlier layer?

Response: Pydantic and Z3 solve different problems. Pydantic checks whether the payload has the expected structure and field types: packages must have ids and weights, robots must have the required identifiers and numeric properties, and the overall payload must match the project's schema. Z3 then reasons about the mathematical relationships between those valid fields. For example, Pydantic can tell me that `capacity` is not a number; Z3 is the component that can tell me that a set of otherwise valid package weights cannot be assigned without violating capacity or battery constraints. That separation keeps parser errors out of the mathematical model and makes the verifier easier to test.

```
payload = VerificationPayload.model_validate(raw_payload)
solver = z3.Solver()
```

Prompt: Can you walk me through the exact order of checks on an LLM response - JSON parsing, action name, Pydantic arguments, then the actual tool - and how that differs from an UNSAT plan?

Response: The runtime first calls `json.loads()` and rejects anything that is not a JSON object. It then checks that the action name is in `ARG_MODELS`. Finally, it instantiates the Pydantic model for that particular action, so each tool gets its own argument schema. `ARG_MODELS` makes the tool interface data-driven: adding or modifying a tool means defining its argument model and registering it, rather than adding another large conditional block with manually written field checks. It also gives the model a typed boundary before project code starts operating on the arguments.

```
step = json.loads(raw)
if entry["action"] not in ARG_MODELS:
    observe(entry, {"error": "Unknown action"}, "syntax")
    continue
```

```
args = ARG_MODELS[entry["action"]](**entry["input"])
```

Prompt: I want to understand the grounding gate properly. If the model changes Pixel's battery from 15 to 95 in the verifier payload, can you show me exactly where the database value wins and what happens next?

Response: The authoritative source is SQLite, not the model's JSON. `grounding_problems()` builds dictionaries from `memory.robots()` and `memory.supplies()`, then compares each robot's capacity, battery, minimum battery, and drain rate field-by-field. It also checks the package weight against the inventory and counts how many units were requested. In the Pixel example, the payload reaches the verifier gate but the comparison sees 95 where SQLite says 15. The agent records a `REJECTED_UNGROUNDED` observation with the specific mismatch and does **not** call Z3 for

that payload. The model then gets that observation as the next piece of context. The reason for doing this before SMT is practical: Z3 is perfectly capable of proving a false proposal if I first let the model lie about the inputs. The grounding gate protects the facts; the SMT layer protects the mathematical relationship between those facts.

```
def grounding_problems(payload):
    """Gate 1: every number the LLM put in the payload must come from the database."""
    fleet = {r["robot_id"]: r for r in memory.robots()}
    stock = {s["item"]: s for s in memory.supplies()}
    problems = []
    for agv in payload["agvs"]:
        row = fleet.get(agv["id"])
        if row is None:
            problems.append(f"unknown robot {agv['id']}")
            continue
        for field in ("capacity", "battery_pct", "min_battery", "drain_rate"):
            if agv[field] != row[field]:
                problems.append(f"{agv['id']} {field} is {agv[field]} but the database says {row[field]}")
    for pkg in payload["packages"]:
        item = pkg["id"].split("#")[0]
        if item not in stock:
            problems.append(f"unknown supply '{item}' (package ids look like 'projector#1')")
        elif pkg["weight"] != stock[item]["weight"]:
            problems.append(f"{pkg['id']} weighs {stock[item]['weight']}kg, not {pkg['weight']}kg")
    for item, count in Counter(p["id"].split("#")[0] for p in payload["packages"]).items():
        if item in stock and count > stock[item]["stock"]:
            problems.append(f"requested {count}x {item} but only {stock[item]['stock']} in stock")
    return problems
```

Prompt: The SMT lab started with a broken capacity comparison and a battery-drain TODO. Can you walk through the two corrections I carried into CampusCourier and explain what the Z3 variables mean?

Response: There are two concrete corrections. First, the capacity constraint has to be `assigned_weight <= capacity`; the supplied lab file had the opposite direction. Second, the battery-drain rule models the remaining battery as `battery_pct - drain_rate * assigned_weight` and requires that value to remain at least the robot's minimum battery. The package decision itself is represented by one integer Z3 variable per package. Its value is an AGV index. The total weight for a robot is then a sum of conditional expressions such as "if this package is assigned to robot 1, add its weight, otherwise add zero." I use `Implies` for the drain rule because an idle robot should not be rejected merely for having no load. The separate low-battery assignment constraint already ensures that a robot below its threshold cannot receive a package. `Implies` makes the load-dependent part of the rule explicit.

```
def battery_drain_constraint(agv: AGVSpec, assigned_weight: "z3.ArithRef") -> "z3.BoolRef":
```

```
    # Battery left after carrying the load must stay at or above the minimum.
    # Only when a load is assigned: an idle AGV below its threshold is already
    # handled by constraint 2 and must not make the whole problem UNSAT.
    # RealVal(str(...)) keeps 0.3 exact (3/10) instead of a binary float.
    battery_left = agv.battery_pct - z3.RealVal(str(agv.drain_rate)) * assigned_weight
    return z3.Implies(assigned_weight > 0, battery_left >= agv.min_battery)
```

```
def verify_dispatch_smt(raw_payload: dict) -> Dict[str, Any]:
    """
    Evaluates payload against First-Order Logic constraints using Z3.
    Returns structured mathematical feedback for the agent context.
    """
    try:
        payload = VerificationPayload.model_validate(raw_payload)
    except Exception as e:
        return {"status": "SYNTACTIC_ERROR", "message": str(e)}
```

```

solver = z3.Solver()
num_agvs = len(payload.agvs)
pkg_vars = {p.id: z3.Int(f"assign_{p.id}") for p in payload.packages}

# 1. Bounds: Every package mapped to a valid AGV index
for p_id, var in pkg_vars.items():
    solver.add(var >= 0, var < num_agvs)

# 2. Battery Theory Check: Low battery AGVs cannot receive assignments
for idx, agv in enumerate(payload.agvs):
    if agv.battery_pct < agv.min_battery:
        for p_id, var in pkg_vars.items():
            solver.add(var != idx)

# 3. Weight Theory Check: Sum of package weights <= AGV capacity
for idx, agv in enumerate(payload.agvs):
    assigned_weight = z3.Sum([
        z3.If(pkg_vars[p.id] == idx, p.weight, 0)
        for p in payload.packages
    ])
    solver.add(assigned_weight <= agv.capacity)

# 4. Battery-Drain Theory Check (Task 4 -- fill in battery_drain_constraint above)
for idx, agv in enumerate(payload.agvs):
    assigned_weight = z3.Sum([
        z3.If(pkg_vars[p.id] == idx, p.weight, 0)
        for p in payload.packages
    ])
    solver.add(battery_drain_constraint(agv, assigned_weight))

# Solve
if solver.check() == z3.sat:
    model = solver.model()
    assignments = {
        p.id: payload.agvs[model[pkg_vars[p.id]].as_long()].id
        for p in payload.packages
    }
    return {
        "status": "SATISFIABLE",
        "assignments": assignments,
        "feedback": "Allocation verified mathematically.",
    }

# ---- UNSAT: figure out WHICH of three distinct failures applies ----
active_agvs = [a for a in payload.agvs if a.battery_pct >= a.min_battery]
disabled_agvs = [a for a in payload.agvs if a.battery_pct < a.min_battery]
total_weight = sum(p.weight for p in payload.packages)
total_active_capacity = sum(a.capacity for a in active_agvs)
max_active_capacity = max((a.capacity for a in active_agvs), default=0)
oversized_packages = [
    p.id for p in payload.packages if p.weight > max_active_capacity
]

```

Prompt: Why did I model one integer variable per package instead of making one Boolean variable for every robot-package combination? What exactly does an assignment like `assign_projector#2` mean?

Response: One variable per package gives the solver a compact assignment representation. The integer value is

the index of the robot in the `payload.agvs` list. So if the variable is 1, package `projector#2` is assigned to the second robot in the payload. The verifier constrains every such variable with `var >= 0` and `var < num_agvs`, which guarantees that every package points to a real robot index. That also means the solver is searching over complete assignments instead of checking an assignment that the LLM has already fixed.

```
pkg_vars = {p.id: z3.Int(f"assign_{p.id}") for p in payload.packages}

for p_id, var in pkg_vars.items():
    solver.add(var >= 0, var < num_agvs)
```

Prompt: Can you derive `assigned_weight` from the `z3.If` expressions for me? I want to understand how the integer package assignments turn into the numeric load used by the constraints.

Response: For a particular robot index, the verifier examines every package variable. If that variable equals the robot's index, the corresponding package weight is included; otherwise zero is included. Summing those conditional values gives the total weight assigned to that robot. That turns the abstract integer assignment into a numeric quantity the other constraints can reason about. The same construction is used once for the capacity constraint and again for the battery-drain constraint. So `assigned_weight` is not another decision variable. It is an arithmetic expression derived from the package assignment variables.

```
assigned_weight = z3.Sum([
    z3.If(pkg_vars[p.id] == idx, p.weight, 0)
    for p in payload.packages
])
solver.add(assigned_weight <= agv.capacity)
```

Prompt: For a robot below `min_battery`, why does the verifier add `var != idx` for every package instead of deleting that robot from the payload first?

Response: Keeping the robot in the solver payload preserves the complete fleet state and lets the mathematical model express why that robot cannot receive anything. The rule is `var != idx` for every package variable when that robot is below the threshold. Deleting the robot would also prevent an assignment, but it would move the responsibility into the caller and change the meaning of the robot list. The current design lets the verifier operate over the supplied fleet while explicitly marking low-battery robots as unavailable for assignment. In the current implementation, the scripted offline engine sometimes filters low-battery robots before constructing the payload, so I should distinguish that optimisation from the hard verifier rule.

```
if agv.battery_pct < agv.min_battery:
    for p_id, var in pkg_vars.items():
        solver.add(var != idx)
```

Prompt: Once `solver.check()` returns SAT, how does the verifier turn the model's integer assignments back into real robot IDs, and why is `as_long()` safe there?

Response: The payload stores the robots in a list, and the integer assignment is an index into that list. After the solver returns SAT, `model[pkg_vars[p.id]].as_long()` gives the integer index, and the verifier uses that index to retrieve the corresponding robot object's `id`. `as_long()` is appropriate because each assignment variable is explicitly an integer and is constrained to a finite integer range. The verifier therefore converts a symbolic solver result back into the human-readable mapping that the commit gate and dashboard use.

```
assignments = {
    p.id: payload.agvs[model[pkg_vars[p.id]].as_long()].id
    for p in payload.packages
}
```

Prompt: When Z3 returns UNSAT, are the total-weight, capacity and oversized-package diagnostics actually coming from an unsat core, or are they computed separately in Python?

Response: They are computed separately in Python after Z3 returns UNSAT. The code looks at the active robots, disabled robots, total package weight, total active capacity, maximum active capacity, and oversized packages to construct a useful explanation. That is not the same as asking Z3 for an unsat core. An unsat core would identify a subset of named solver constraints whose combination caused inconsistency. The current project instead gives

the LLM structured, hand-written diagnostics that are useful for changing the request. This distinction matters because the project can truthfully say it gets a mathematical UNSAT result from Z3, but it should not claim that the explanation is a formal Z3 unsat core.

```
if solver.check() == z3.unsat:
    active_agvs = [a for a in payload.agvs if a.battery_pct >= a.min_battery]
    total_weight = sum(p.weight for p in payload.packages)
    total_active_capacity = sum(a.capacity for a in active_agvs)
    # Python then builds the diagnostics dictionary
```

Prompt: I want to be very clear about the division of responsibility. Is Z3 actually checking the path, or is A* doing that part while Z3 proves the allocation constraints?

Response: Z3 and A* are proving different parts of the plan. For navigation, `world.make_nav_env()` takes the shared ASCII map and changes quiet-zone cells (g) into blocked cells when the robot does not have quiet wheels. A* is then run on that robot-specific grid. So the route planner literally has no legal path through those cells. Z3, by contrast, is proving the allocation constraints encoded in `core/verifier.py`: valid robot indices, battery eligibility, capacity, and load-dependent battery drain. The quiet-zone path rule is not represented as a route decision variable in the SMT model. The honest statement is therefore: *the allocation is mathematically verified by Z3, while the route is independently planned and checked by A**. The current commit gate requires both pieces before a dispatch is recorded.

```
sys.path.insert(0, str(Path(__file__).parent / "core"))
```

```
from environment import SurgicallabEnvironment # Lab 1 (A*)
from planner import ForkliftPlanner # Lab 1 (A*)
```

```
CAMPUS_MAP = """
#####
#B...#.....#.....#
#.#.#.#.#####.#.####.#
#.#...~..#.....#..P.#.#
#.#.####.#.#.####.#.#
#...#...#...#...#...#
###.#.####.#.####.###.#
#.....#...#...#.....#
#...#.#.#####S####.#
#...#.....gggg.....#
#.#.####.#gggg#####.#
#.#.....#.....X#
#####
"""
```

```
ROWS = CAMPUS_MAP.strip().splitlines()
HEIGHT, WIDTH = len(ROWS), len(ROWS[0])
```

```
LOCATION_CODES = {"B": "Supply Room", "X": "Studio 3", "P": "Library", "S": "Corridor B"}
```

```
def cells(char):
    return {(x, y) for y, row in enumerate(ROWS) for x, c in enumerate(row) if c == char}
```

```
LOCATIONS = {name: next(iter(cells(code))) for code, name in LOCATION_CODES.items()}
WET = cells("~")
GAS = cells("g")
```

```

def make_nav_env(avoid_quiet=False):
    """Lab 1's environment with the campus geometry swapped in. Robots that aren't
    quiet-wheeled see the quiet zone as solid rock, so A* routes around it."""
    env = SurgicallabEnvironment(map_type="trivial") # blank preset; geometry replaced below
    env.width, env.height = WIDTH, HEIGHT
    env.grid = [[1 if c == "#" or (avoid_quiet and c == "g") else 0 for c in row] for row in ROWS]
    return env

def plan_route(start, goal, quiet_wheels=True):
    env = make_nav_env(avoid_quiet=not quiet_wheels)
    path, expanded, seconds = ForkliftPlanner(env).a_star(tuple(start), tuple(goal), "manhattan")
    return {
        "reachable": bool(path),
        "path": [list(p) for p in path],
        "steps": max(len(path) - 1, 0),
        "nodes_expanded": expanded,
    }

```

Prompt: Why does every verification attempt create a fresh `z3.Solver()`? What would get confusing if I reused one solver across retries?

Response: Each verification call should represent one candidate allocation problem. A fresh solver prevents constraints from an earlier rejected payload carrying into the next one. Retries are deliberately different candidate problems, so keeping solver state separate makes the behaviour deterministic and much easier to reason about.

Prompt: Why should I test `verify_dispatch_smt()` directly with hand-built SAT and UNSAT payloads if the normal application gets to it through the LLM anyway?

Response: Direct verifier tests isolate the mathematical layer from model behaviour. I can construct a payload with known weights and robot limits and assert the expected result without asking an LLM to generate it. This makes failures easier to localise: a direct failure points to the verifier, while an end-to-end failure can involve the model or integration.

```

if __name__ == "__main__":
    # Quick manual sanity check -- python verifier.py
    import json

    sat_case = {
        "packages": [{"id": "Item-1", "weight": 10}, {"id": "Item-2", "weight": 15}],
        "agvs": [
            {"id": "AGV-A", "capacity": 30, "battery_pct": 80, "min_battery": 25},
            {"id": "AGV-B", "capacity": 20, "battery_pct": 50, "min_battery": 25},
        ],
    }
    unsat_case = {
        "packages": [
            {"id": "Heavy-Box-1", "weight": 40},
            {"id": "Heavy-Box-2", "weight": 25},
        ],
        "agvs": [
            {"id": "AGV-1", "capacity": 30, "battery_pct": 90, "min_battery": 25},
            {"id": "AGV-2", "capacity": 20, "battery_pct": 10, "min_battery": 25},
        ],
    }

    print("SAT case:")
    print(json.dumps(verify_dispatch_smt(sat_case), indent=2))

```

Prompt: Can we go through the overload preset numerically - 3 projectors plus the whiteboard - and follow the first UNSAT, the changed retry, and the exact point where repeating the same rejected payload gets blocked?

Response: The request is 3 projectors at 12 kg each plus one whiteboard at 18 kg, so the total requested load is 54 kg. Pixel is already below its minimum battery, so the active robots are Atlas at 30 kg capacity and Comet at 20 kg capacity: 50 kg of usable raw capacity between them. The first payload therefore cannot be satisfied. The model receives Z3's UNSAT result and structured diagnostics. The scripted overload story then removes the whiteboard and retries with 3 projectors, which is 36 kg. That retry is genuinely different; it is not a second copy of the original payload. There is also an explicit memory of rejected payloads: the code serialises the payload with sorted JSON and stores it in `unsat_payloads`. If the exact same payload appears again, it is marked `BLOCKED_REPEAT` and Z3 is not called again. This turns the “read the UNSAT result and change the plan” rule into an actual execution rule rather than a prompt-only instruction.

```

except (ValidationError, TypeError) as e:
    observe(entry, {"error": str(e)}, "syntax")
    continue

action = entry["action"]
if action == "finish":
    refusal = None
    if args.assignments:
        if not last_verdict or last_verdict.get("status") != "SATISFIABLE":
            refusal = "BLOCKED: no SATISFIABLE verification backs these assignments."
        elif args.assignments != last_verdict["assignments"]:
            refusal = "BLOCKED: assignments differ from the Z3-verified model."
        elif args.destination not in world.LOCATIONS:
            refusal = f"BLOCKED: unknown destination {args.destination}."
        else:
            missing = [r for r in set(args.assignments.values())
                        if routes.get(r, {}).get("destination") != args.destination]
            if missing:
                refusal = f"BLOCKED: plan_route to {args.destination} first for {missing}."
    if refusal:
        observe(entry, {"error": refusal}, "blocked")
        continue
    observe(entry, {"status": "COMMITTED" if args.assignments else "NO DISPATCH"}, "finish")
    return {"ok": bool(args.assignments), "final_answer": args.final_answer,
            "destination": args.destination, "assignments": args.assignments,
            "packages": last_payload["packages"] if args.assignments else [],
            "routes": routes, "trace": trace, "messages": messages}

if action == "verify_dispatch_smt":
    payload = args.model_dump()
    key = json.dumps(payload, sort_keys=True)
    problems = grounding_problems(payload)
    if problems:
        observe(entry, {"status": "REJECTED_UNGROUNDED", "problems": problems}, "blocked")
    elif key in unsat_payloads:
        observe(entry, {"status": "BLOCKED_REPEAT",
                        "feedback": "This exact payload was already UNSATISFIABLE. Change it."}, "b
    else:
        last_verdict, last_payload = verify_dispatch_smt(payload), payload
        if last_verdict["status"] != "SATISFIABLE":
            unsat_payloads.add(key)
        observe(entry, last_verdict, "sat" if last_verdict["status"] == "SATISFIABLE" else "unsat")
elif action == "plan_route":
    result = plan_route_tool(args.robot_id, args.destination)
    if result.get("reachable"):
        routes[args.robot_id] = result

```

Prompt: I want to check a nasty edge case before I call the verifier complete. If the same package ID appears twice,

does the current code explicitly reject it, or is uniqueness just assumed by the normal prompts and scripts?

Response: The current verifier uses the package ID as the key of the `pkg_vars` dictionary. That means duplicate IDs are not explicitly rejected as a dedicated validation error. The model prompt tells the LLM to create identifiers such as `projector#1`, `projector#2`, so the normal project stories produce unique ids, but the hard verifier itself does not have a separate “all package ids must be unique” constraint. That is a genuine edge case I would record as a test gap rather than claiming the verifier is complete. The clean fix would be to validate uniqueness before constructing the solver variables, for example by comparing `len(ids)` with `len(set(ids))`. The fact that I can identify this from the code is also why I prefer a practical review log over a purely theoretical explanation.

Prompt: For the final step, can you show me why `finish()` cannot just trust the model? What happens when its assignments do not match the last SAT result or when it has not planned a route for an assigned robot?

Response: The finish action is treated as a guarded transition. If the model supplies assignments, the code first checks that there was a latest `SATISFIABLE` verdict. It then checks that the supplied assignment dictionary is exactly the same as the solver’s assignment. It also checks that the destination is a known location and that every robot named in the assignment has a recorded route to that destination. If any of those conditions fail, the action is marked `blocked`, an error is sent back as an observation, and the loop continues. Nothing is committed. This is an important implementation detail because a system prompt is advice to the LLM. The finish gate is executable policy.

```
action = entry["action"]
if action == "finish":
    refusal = None
    if args.assignments:
        if not last_verdict or last_verdict.get("status") != "SATISFIABLE":
            refusal = "BLOCKED: no SATISFIABLE verification backs these assignments."
        elif args.assignments != last_verdict["assignments"]:
            refusal = "BLOCKED: assignments differ from the Z3-verified model."
        elif args.destination not in world.LOCATIONS:
            refusal = f"BLOCKED: unknown destination {args.destination}."
        else:
            missing = [r for r in set(args.assignments.values())
                        if routes.get(r, {}).get("destination") != args.destination]
            if missing:
                refusal = f"BLOCKED: plan_route to {args.destination} first for {missing}."
    if refusal:
        observe(entry, {"error": refusal}, "blocked")
        continue
    observe(entry, {"status": "COMMITTED" if args.assignments else "NO DISPATCH"}, "finish")
    return {"ok": bool(args.assignments), "final_answer": args.final_answer,
            "destination": args.destination, "assignments": args.assignments,
            "packages": last_payload["packages"] if args.assignments else [],
            "routes": routes, "trace": trace, "messages": messages}

if action == "verify_dispatch_smt":
    payload = args.model_dump()
    key = json.dumps(payload, sort_keys=True)
    problems = grounding_problems(payload)
    if problems:
        observe(entry, {"status": "REJECTED_UNGROUNDED", "problems": problems}, "blocked")
    elif key in unsat_payloads:
        observe(entry, {"status": "BLOCKED_REPEAT",
                        "feedback": "This exact payload was already UNSATISFIABLE. Change it."}, "b
    else:
        last_verdict, last_payload = verify_dispatch_smt(payload), payload
```